DSA Interview Guide

Patterns + 500 problems with C# solutions

Theory + practical + real-world scenario questions, with solutions

Interview Prep Series | Prepared for Mustafa

Contents

1. Complexity Analysis and Problem-Solving Approach 4
2. Arrays and Hashing 5
3. Two Pointers 8
4. Sliding Window 9
5. Prefix Sum 11
6. Binary Search 12
7. Sorting 14
8. Strings 15
9. Linked Lists 17
10. Stacks and Queues 19
11. Recursion and Backtracking 20
12. Trees and Binary Search Trees 21
13. Heaps and Priority Queues 23
14. Graphs 24
15. Dynamic Programming 26
16. Greedy and Intervals 28
17. Tries and Bit Manipulation 29
18. System-Flavoured DSA and Real-World Scenarios 30
19. Quick-Fire Questions 31
20. Arrays: More Interview Problems 32
21. Strings: More Interview Problems 36
22. Hashing, Two Pointers and Sliding Window (More) 39
23. Binary Search (More) 41
24. Linked Lists (More) 43
25. Stacks and Queues (More) 45
26. Recursion and Backtracking (More) 48
27. Trees (More) 50
28. Heaps (More) 54
29. Graphs (More) 56
30. Dynamic Programming (More) 59
31. Greedy (More) 63
32. Math and Bit Manipulation (More) 65

33. Design Data Structures **67**

1. Complexity Analysis and Problem-Solving Approach

Q1. What is Big-O notation? List common complexities.

Big-O describes how time or memory grows with input size n in the worst case, ignoring constants. From fastest to slowest: O(1) hash lookup, O(log n) binary search, O(n) single loop, O(n log n) good sorting, O(n squared) nested loops, O(2^n) subsets, O(n!) permutations. Big-Omega is the lower bound and Big-Theta the tight bound; interviewers usually mean Big-O.

> **Interview tip:** Rough limits for a 1-second judge: n up to 10^8 for O(n), 10^6 for O(n log n), 10^4 for O(n squared), 20-25 for O(2^n).

Q2. How should you approach a DSA problem in an interview?

- Clarify inputs, constraints, edge cases (empty, duplicates, negatives, overflow).
- Give a brute-force solution and its complexity first.
- Spot the pattern (two pointers, sliding window, hashing, binary search, BFS/DFS, DP, heap).
- Explain the optimised approach before coding; then code cleanly.
- Dry run with a small example; state final time and space complexity.

Q3. What is amortised complexity?

Average cost per operation over a sequence. `List<T>.Add` is O(1) amortised: occasionally the array doubles (O(n)) but over n adds the total is O(n).

Q4. What is the space complexity of recursion?

The call stack depth. Recursive DFS on a skewed tree uses O(n) stack; on a balanced tree O(log n). Deep recursion can cause StackOverflowException; convert to an explicit stack if depth can be large.

Q5. Pattern recognition cheat sheet.

- Sorted array, find pair/target -> two pointers or binary search.
- Contiguous subarray/substring with a condition -> sliding window.
- Subarray sum equals K -> prefix sum + hash map.
- "Top K", "Kth largest", merge K lists -> heap.
- Next greater/smaller element -> monotonic stack.
- Shortest path in unweighted graph/grid -> BFS.
- All combinations/permutations -> backtracking.
- Overlapping subproblems with optimal choice -> dynamic programming.
- Intervals -> sort by start then merge.
- Prefix search on words -> trie.
- Connected components / grouping -> DFS or Union-Find.
- Answer is monotonic ("minimum capacity such that...") -> binary search on answer.

2. Arrays and Hashing

Q6. Two Sum: return indices of two numbers adding to target.

Brute force O(n squared). Optimal: one pass with a hash map from value to index. O(n) time, O(n) space.

```
int[] TwoSum(int[] nums, int target) {
    var seen = new Dictionary<int, int>();
    for (int i = 0; i < nums.Length; i++) {
        if (seen.TryGetValue(target - nums[i], out int j)) return [j, i];
        seen[nums[i]] = i;
    }
    return [];
}
```

> **Interview tip:** Real-world: matching a debit with a credit that sums to a settlement amount.

Q7. Contains Duplicate.

```
bool ContainsDuplicate(int[] nums) { var s = new HashSet<int>(); foreach (var n in nums) if (!s.Add(n))
    return true; return false; }
```

O(n) time, O(n) space. Alternative: sort O(n log n), O(1) extra.

Q8. Valid Anagram.

```
bool IsAnagram(string s, string t) {
    if (s.Length != t.Length) return false;
    var c = new int[26];
    for (int i = 0; i < s.Length; i++) { c[s[i] - 'a']++; c[t[i] - 'a']--; }
    return c.All(x => x == 0);
}
```

Q9. Group Anagrams.

Key each word by its sorted characters (or 26-count signature). O(n k log k).

```
IList<IList<string>> GroupAnagrams(string[] strs) {
    var map = new Dictionary<string, IList<string>>();
    foreach (var s in strs) {
        var a = s.ToCharArray(); Array.Sort(a); var key = new string(a);
        if (!map.TryGetValue(key, out var list)) map[key] = list = new List<string>();
        list.Add(s);
    }
    return map.Values.ToList();
}
```

Q10. Top K Frequent Elements.

Count with a dictionary, then bucket sort by frequency: O(n). (Heap gives O(n log k).)

```
int[] TopKFrequent(int[] nums, int k) {
    var freq = new Dictionary<int, int>();
    foreach (var n in nums) freq[n] = freq.GetValueOrDefault(n) + 1;
    var buckets = new List<int>[nums.Length + 1];
    foreach (var (n, f) in freq) (buckets[f] ??= new()).Add(n);
    var res = new List<int>();
    for (int f = buckets.Length - 1; f > 0 && res.Count < k; f--)
        if (buckets[f] != null) res.AddRange(buckets[f]);
    return res.Take(k).ToArray();
}
```

Q11. Product of Array Except Self (no division).

Prefix products left-to-right, then multiply by suffix products right-to-left. O(n) time, O(1) extra.

```
int[] ProductExceptSelf(int[] nums) {
    int n = nums.Length; var res = new int[n]; res[0] = 1;
    for (int i = 1; i < n; i++) res[i] = res[i - 1] * nums[i - 1];
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) { res[i] *= suffix; suffix *= nums[i]; }
    return res;
}
```

Q12. Longest Consecutive Sequence in O(n).

Put all in a HashSet; start counting only from numbers whose predecessor is absent.

```
int LongestConsecutive(int[] nums) {
    var set = new HashSet<int>(nums); int best = 0;
    foreach (var n in set) {
        if (set.Contains(n - 1)) continue;
        int len = 1; while (set.Contains(n + len)) len++;
        best = Math.Max(best, len);
    }
    return best;
}
```

Q13. Maximum Subarray (Kadane's algorithm).

At each element, either extend the current subarray or start fresh. O(n), O(1).

```
int MaxSubArray(int[] a) {
    int cur = a[0], best = a[0];
    for (int i = 1; i < a.Length; i++) { cur = Math.Max(a[i], cur + a[i]); best = Math.Max(best, cur); }
    return best;
}
```

Q14. Best Time to Buy and Sell Stock (one transaction).

Track the minimum price so far and the best profit.

```
int MaxProfit(int[] p) { int min = int.MaxValue, best = 0;
    foreach (var x in p) { min = Math.Min(min, x); best = Math.Max(best, x - min); } return best; }
```

Unlimited transactions: sum all positive day-to-day differences.

Q15. Move Zeroes to the end keeping order.

```
void MoveZeroes(int[] a) { int w = 0;
    for (int r = 0; r < a.Length; r++) if (a[r] != 0) { (a[w], a[r]) = (a[r], a[w]); w++; } }
```

Q16. Rotate an array by k steps.

Reverse all, reverse first k, reverse the rest. O(n), O(1).

```
void Rotate(int[] a, int k) { k %= a.Length;
    Array.Reverse(a); Array.Reverse(a, 0, k); Array.Reverse(a, k, a.Length - k); }
```

Q17. Majority Element (Boyer-Moore voting).

```
int Majority(int[] a) { int cand = 0, cnt = 0;
    foreach (var x in a) { if (cnt == 0) cand = x; cnt += x == cand ? 1 : -1; } return cand; }
```

Q18. Find missing number from 0..n.

Sum formula (watch overflow) or XOR of all indices and values.

```
int Missing(int[] a) { int x = a.Length; for (int i = 0; i < a.Length; i++) x ^= i ^ a[i]; return x; }
```

Q19. Sort Colors (Dutch National Flag) - sort 0s, 1s, 2s in one pass.

```
void SortColors(int[] a) { int lo = 0, mid = 0, hi = a.Length - 1;
    while (mid <= hi) {
        if (a[mid] == 0) { (a[lo], a[mid]) = (a[mid], a[lo]); lo++; mid++; }
        else if (a[mid] == 1) mid++;
        else { (a[mid], a[hi]) = (a[hi], a[mid]); hi--; }
    } }
```

Q20. Merge two sorted arrays in place (nums1 has extra space).

Fill from the end to avoid overwriting.

```
void Merge(int[] a, int m, int[] b, int n) { int i = m - 1, j = n - 1, k = m + n - 1;
    while (j >= 0) a[k--] = (i >= 0 && a[i] > b[j]) ? a[i--] : b[j--]; }
```

Q21. Find the duplicate number (array of n+1 numbers in 1..n) without modifying it.

Floyd's cycle detection treating values as next pointers. O(n), O(1).

```
int FindDuplicate(int[] a) {
    int slow = a[0], fast = a[0];
    do { slow = a[slow]; fast = a[a[fast]]; } while (slow != fast);
    slow = a[0]; while (slow != fast) { slow = a[slow]; fast = a[fast]; }
    return slow;
}
```

Q22. Spiral Matrix traversal.

```
IList<int> Spiral(int[][] m) {
    var r = new List<int>(); int top = 0, bot = m.Length - 1, l = 0, rt = m[0].Length - 1;
    while (top <= bot && l <= rt) {
        for (int j = l; j <= rt; j++) r.Add(m[top][j]); top++;
        for (int i = top; i <= bot; i++) r.Add(m[i][rt]); rt--;
        if (top <= bot) { for (int j = rt; j >= l; j--) r.Add(m[bot][j]); bot--; }
        if (l <= rt) { for (int i = bot; i >= top; i--) r.Add(m[i][l]); l++; }
    }
    return r;
}
```

Q23. Rotate an N x N matrix 90 degrees clockwise in place.

Transpose, then reverse each row.

```
void RotateMatrix(int[][] m) { int n = m.Length;
    for (int i = 0; i < n; i++) for (int j = i + 1; j < n; j++) (m[i][j], m[j][i]) = (m[j][i], m[i][j]);
    foreach (var row in m) Array.Reverse(row); }
```

3. Two Pointers

Q24. Valid Palindrome (ignore non-alphanumerics).

Two pointers from both ends, skip non-alphanumeric, compare lowercased. O(n), O(1).

```
bool IsPal(string s) { int i = 0, j = s.Length - 1;
    while (i < j) {
        while (i < j && !char.IsLetterOrDigit(s[i])) i++;
        while (i < j && !char.IsLetterOrDigit(s[j])) j--;
        if (char.ToLower(s[i++]) != char.ToLower(s[j--])) return false;
    } return true; }
```

Q25. Two Sum II on a sorted array.

```
int[] TwoSumSorted(int[] a, int t) { int i = 0, j = a.Length - 1;
    while (i < j) { int s = a[i] + a[j]; if (s == t) return [i, j]; if (s < t) i++; else j--; }
    return []; }
```

Q26. 3Sum: all unique triplets summing to zero.

Sort, fix one element, two-pointer the rest, skip duplicates. O(n squared).

```
IList<IList<int>> ThreeSum(int[] a) {
    Array.Sort(a); var res = new List<IList<int>>();
    for (int i = 0; i < a.Length - 2; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;
        int l = i + 1, r = a.Length - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s < 0) l++; else if (s > 0) r--;
            else { res.Add([a[i], a[l], a[r]]); while (l < r && a[l] == a[l + 1]) l++; while (l < r && a[
    r] == a[r - 1]) r--; l++; r--; }
        }
    }
    return res;
}
```

Q27. Container With Most Water.

Move the pointer at the shorter line inward (the taller can't help with a shorter limit).

```
int MaxArea(int[] h) { int i = 0, j = h.Length - 1, best = 0;
    while (i < j) { best = Math.Max(best, Math.Min(h[i], h[j]) * (j - i)); if (h[i] < h[j]) i++; else j--;
     }
    return best; }
```

Q28. Trapping Rain Water.

Two pointers with leftMax and rightMax; process the smaller side. O(n), O(1).

```
int Trap(int[] h) { int l = 0, r = h.Length - 1, lm = 0, rm = 0, w = 0;
    while (l < r) {
        if (h[l] < h[r]) { lm = Math.Max(lm, h[l]); w += lm - h[l++]; }
        else { rm = Math.Max(rm, h[r]); w += rm - h[r--]; }
    } return w; }
```

Q29. Remove duplicates from a sorted array in place.

```
int RemoveDuplicates(int[] a) { if (a.Length == 0) return 0; int w = 1;
    for (int i = 1; i < a.Length; i++) if (a[i] != a[w - 1]) a[w++] = a[i]; return w; }
```

4. Sliding Window

Q30. Longest Substring Without Repeating Characters.

Expand right; when a repeat appears, jump left past the previous occurrence. O(n).

```
int LengthOfLongestSubstring(string s) {
    var last = new Dictionary<char, int>(); int l = 0, best = 0;
    for (int r = 0; r < s.Length; r++) {
        if (last.TryGetValue(s[r], out int p) && p >= l) l = p + 1;
        last[s[r]] = r; best = Math.Max(best, r - l + 1);
    }
    return best;
}
```

Q31. Maximum sum of a subarray of size k (fixed window).

```
int MaxSumK(int[] a, int k) { int s = 0; for (int i = 0; i < k; i++) s += a[i]; int best = s;
    for (int i = k; i < a.Length; i++) { s += a[i] - a[i - k]; best = Math.Max(best, s); } return best; }
```

Q32. Longest Repeating Character Replacement (at most k changes).

Window is valid while `windowLen - maxFreq <= k`.

```
int CharacterReplacement(string s, int k) {
    var c = new int[26]; int l = 0, maxF = 0, best = 0;
    for (int r = 0; r < s.Length; r++) {
        maxF = Math.Max(maxF, ++c[s[r] - 'A']);
        while (r - l + 1 - maxF > k) c[s[l++] - 'A']--;
        best = Math.Max(best, r - l + 1);
    }
    return best;
}
```

Q33. Minimum Window Substring.

Expand until all required chars are covered, then shrink from the left while still valid. O(m + n).

```
string MinWindow(string s, string t) {
    var need = new Dictionary<char, int>();
    foreach (var ch in t) need[ch] = need.GetValueOrDefault(ch) + 1;
    int missing = t.Length, l = 0, start = 0, len = int.MaxValue;
    for (int r = 0; r < s.Length; r++) {
        if (need.ContainsKey(s[r]) && need[s[r]]-- > 0) missing--;
        while (missing == 0) {
            if (r - l + 1 < len) { len = r - l + 1; start = l; }
            if (need.ContainsKey(s[l]) && ++need[s[l]] > 0) missing++;
            l++;
        }
    }
    return len == int.MaxValue ? "" : s.Substring(start, len);
}
```

Q34. Permutation in String (does s2 contain an anagram of s1?).

Fixed window of size s1.Length with count arrays; compare counts as you slide.

```
bool CheckInclusion(string s1, string s2) {
    if (s1.Length > s2.Length) return false;
    var a = new int[26]; var b = new int[26];
    for (int i = 0; i < s1.Length; i++) { a[s1[i] - 'a']++; b[s2[i] - 'a']++; }
    for (int i = s1.Length; ; i++) {
        if (a.SequenceEqual(b)) return true;
        if (i == s2.Length) return false;
        b[s2[i] - 'a']++; b[s2[i - s1.Length] - 'a']--;
    }
}
```

Q35. Sliding Window Maximum.

Monotonic deque storing indices of decreasing values. O(n).

```
int[] MaxSlidingWindow(int[] a, int k) {
    var dq = new LinkedList<int>(); var res = new int[a.Length - k + 1];
    for (int i = 0; i < a.Length; i++) {
        if (dq.Count > 0 && dq.First!.Value <= i - k) dq.RemoveFirst();
        while (dq.Count > 0 && a[dq.Last!.Value] <= a[i]) dq.RemoveLast();
        dq.AddLast(i);
        if (i >= k - 1) res[i - k + 1] = a[dq.First!.Value];
    }
    return res;
}
```

5. Prefix Sum

Q36. Subarray Sum Equals K (with negatives).

Count prefix sums seen; a previous prefix of `sum - k` means a valid subarray ends here. O(n).

```
int SubarraySum(int[] a, int k) {
    var cnt = new Dictionary<int, int> { [0] = 1 }; int sum = 0, res = 0;
    foreach (var x in a) { sum += x; res += cnt.GetValueOrDefault(sum - k); cnt[sum] = cnt.
    GetValueOrDefault(sum) + 1; }
    return res;
}
```

> **Interview tip:** Real-world: finding a run of transactions that adds up to a disputed settlement difference.

Q37. Range Sum Query (immutable).

Precompute `pre[i+1] = pre[i] + a[i]`; sum(l, r) = `pre[r+1] - pre[l]`. O(1) per query.

Q38. Find the pivot (equilibrium) index.

```
int PivotIndex(int[] a) { int total = a.Sum(), left = 0;
    for (int i = 0; i < a.Length; i++) { if (left == total - left - a[i]) return i; left += a[i]; }
    return -1; }
```

6. Binary Search

Q39. Classic binary search; why `lo + (hi - lo) / 2`?

`(lo + hi) / 2` can overflow int for large indices.

```
int Search(int[] a, int t) { int lo = 0, hi = a.Length - 1;
    while (lo <= hi) { int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid; if (a[mid] < t) lo = mid + 1; else hi = mid - 1; }
    return -1; }
```

Q40. First and last position of an element.

Run lower-bound twice: first index of t, and first index of t + 1 minus one.

```
int LowerBound(int[] a, int t) { int lo = 0, hi = a.Length;
    while (lo < hi) { int m = lo + (hi - lo) / 2; if (a[m] < t) lo = m + 1; else hi = m; } return lo; }
int[] SearchRange(int[] a, int t) { int f = LowerBound(a, t);
    if (f == a.Length || a[f] != t) return [-1, -1]; return [f, LowerBound(a, t + 1) - 1]; }
```

Q41. Search in a Rotated Sorted Array.

One half is always sorted; check if the target lies in that half.

```
int SearchRotated(int[] a, int t) { int lo = 0, hi = a.Length - 1;
    while (lo <= hi) { int m = lo + (hi - lo) / 2; if (a[m] == t) return m;
        if (a[lo] <= a[m]) { if (a[lo] <= t && t < a[m]) hi = m - 1; else lo = m + 1; }
        else { if (a[m] < t && t <= a[hi]) lo = m + 1; else hi = m - 1; } }
    return -1; }
```

Q42. Find Minimum in a Rotated Sorted Array.

```
int FindMin(int[] a) { int lo = 0, hi = a.Length - 1;
    while (lo < hi) { int m = lo + (hi - lo) / 2; if (a[m] > a[hi]) lo = m + 1; else hi = m; } return a[
    lo]; }
```

Q43. Koko Eating Bananas / binary search on the answer.

If a speed works, every higher speed works (monotonic), so binary search the smallest valid speed.

```
int MinEatingSpeed(int[] piles, int h) { int lo = 1, hi = piles.Max();
    while (lo < hi) { int k = lo + (hi - lo) / 2; long hours = 0;
        foreach (var p in piles) hours += (p + k - 1) / k;
        if (hours <= h) hi = k; else lo = k + 1; }
    return lo; }
```

Same pattern: ship packages within D days, split array largest sum, minimum time to finish jobs.

Q44. Search a 2D matrix (rows sorted, each row starts after the previous ends).

Treat as a flattened sorted array: index i maps to `m[i / cols][i % cols]`. O(log(mn)).

Q45. Median of Two Sorted Arrays (hard).

Binary search the partition in the smaller array so left halves contain half the elements and maxLeft <= minRight. O(log(min(m, n))).

```
double FindMedian(int[] a, int[] b) {
    if (a.Length > b.Length) return FindMedian(b, a);
    int m = a.Length, n = b.Length, lo = 0, hi = m, half = (m + n + 1) / 2;
    while (lo <= hi) {
        int i = (lo + hi) / 2, j = half - i;
        int aL = i == 0 ? int.MinValue : a[i - 1], aR = i == m ? int.MaxValue : a[i];
        int bL = j == 0 ? int.MinValue : b[j - 1], bR = j == n ? int.MaxValue : b[j];
        if (aL <= bR && bL <= aR)
            return (m + n) % 2 == 1 ? Math.Max(aL, bL) : (Math.Max(aL, bL) + Math.Min(aR, bR)) / 2.0;
        if (aL > bR) hi = i - 1; else lo = i + 1;
    }
    throw new ArgumentException();
}
```

7. Sorting

Q46. Compare common sorting algorithms.

- Bubble/Selection/Insertion: O(n squared); insertion is good for nearly-sorted small data.
- Merge sort: O(n log n) always, stable, O(n) extra space; good for linked lists and external sorting of huge files.
- Quick sort: O(n log n) average, O(n squared) worst (bad pivots), in place, not stable.
- Heap sort: O(n log n), in place, not stable.
- Counting/Radix: O(n + k) for integers in a small range.
- .NET `Array.Sort` uses introsort (quick + heap + insertion), unstable; LINQ `OrderBy` is stable.

Q47. Implement merge sort.

```
void MergeSort(int[] a, int l, int r) {
    if (l >= r) return; int m = (l + r) / 2;
    MergeSort(a, l, m); MergeSort(a, m + 1, r);
    var tmp = new int[r - l + 1]; int i = l, j = m + 1, k = 0;
    while (i <= m && j <= r) tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
    while (i <= m) tmp[k++] = a[i++]; while (j <= r) tmp[k++] = a[j++];
    Array.Copy(tmp, 0, a, l, tmp.Length);
}
```

Q48. Implement quick sort (Lomuto partition).

```
void QuickSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int p = a[hi], i = lo;
    for (int j = lo; j < hi; j++) if (a[j] < p) { (a[i], a[j]) = (a[j], a[i]); i++; }
    (a[i], a[hi]) = (a[hi], a[i]);
    QuickSort(a, lo, i - 1); QuickSort(a, i + 1, hi);
}
```

Use a random pivot to avoid worst case on sorted input.

Q49. What does "stable sort" mean and why does it matter?

Equal elements keep their original relative order. Matters for multi-key sorts: sort by time, then stable sort by merchant keeps time order within each merchant.

Q50. Kth Largest Element.

Min-heap of size k: O(n log k). Quickselect: O(n) average.

```
int FindKthLargest(int[] nums, int k) {
    var pq = new PriorityQueue<int, int>();
    foreach (var n in nums) { pq.Enqueue(n, n); if (pq.Count > k) pq.Dequeue(); }
    return pq.Peek();
}
```

8. Strings

Q51. Reverse words in a sentence.

```
string ReverseWords(string s) => string.Join(' ', s.Split(' ', StringSplitOptions.RemoveEmptyEntries).
    Reverse());
```

Q52. Longest Common Prefix.

```
string LongestCommonPrefix(string[] s) { if (s.Length == 0) return "";
    var p = s[0]; foreach (var w in s) while (!w.StartsWith(p)) p = p[..^1]; return p; }
```

Q53. Valid Parentheses.

```
bool IsValid(string s) { var st = new Stack<char>();
    foreach (var c in s) {
        if (c is '(' or '[' or '{') st.Push(c);
        else { if (st.Count == 0) return false; var o = st.Pop();
            if ((c == ')' && o != '(') || (c == ']' && o != '[') || (c == '}' && o != '{')) return false;
    }
    } return st.Count == 0; }
```

Q54. Longest Palindromic Substring.

Expand around each centre (odd and even). O(n squared), O(1).

```
string LongestPalindrome(string s) { int start = 0, len = 0;
    for (int c = 0; c < s.Length; c++)
        foreach (var (l0, r0) in new[] { (c, c), (c, c + 1) }) {
            int l = l0, r = r0;
            while (l >= 0 && r < s.Length && s[l] == s[r]) { l--; r++; }
            if (r - l - 1 > len) { len = r - l - 1; start = l + 1; }
        }
    return s.Substring(start, len); }
```

Q55. String compression ("aaabb" -> "a3b2").

```
string Compress(string s) { var sb = new StringBuilder();
    for (int i = 0; i < s.Length; ) { int j = i; while (j < s.Length && s[j] == s[i]) j++;
        sb.Append(s[i]).Append(j - i); i = j; }
    return sb.Length < s.Length ? sb.ToString() : s; }
```

Q56. String to Integer (atoi) with overflow handling.

Skip spaces, read sign, read digits, clamp to int range before overflowing.

```
int MyAtoi(string s) { int i = 0, sign = 1; long r = 0;
    while (i < s.Length && s[i] == ' ') i++;
    if (i < s.Length && (s[i] == '+' || s[i] == '-')) sign = s[i++] == '-' ? -1 : 1;
    while (i < s.Length && char.IsDigit(s[i])) { r = r * 10 + (s[i++] - '0');
        if (r * sign > int.MaxValue) return int.MaxValue; if (r * sign < int.MinValue) return int.
    MinValue; }
    return (int)(r * sign); }
```

Q57. What is the KMP algorithm (pattern search)?

Builds an LPS (longest proper prefix which is also suffix) array for the pattern so mismatches don't restart from scratch. Search in O(n + m) instead of O(nm). Alternative: Rabin-Karp rolling hash.

```
int StrStr(string text, string pat) {
    var lps = new int[pat.Length];
    for (int i = 1, len = 0; i < pat.Length; ) {
        if (pat[i] == pat[len]) lps[i++] = ++len; else if (len > 0) len = lps[len - 1]; else lps[i++] = 0;
     }
    for (int i = 0, j = 0; i < text.Length; ) {
        if (text[i] == pat[j]) { i++; j++; if (j == pat.Length) return i - j; }
        else if (j > 0) j = lps[j - 1]; else i++; }
    return pat.Length == 0 ? 0 : -1;
}
```

9. Linked Lists

Q58. Reverse a linked list (iterative and recursive).

```
public class ListNode(int val, ListNode? next = null) { public int val = val; public ListNode? next =
    next; }

ListNode? Reverse(ListNode? head) { ListNode? prev = null;
    while (head != null) { var nxt = head.next; head.next = prev; prev = head; head = nxt; } return prev;
    }

ListNode? ReverseRec(ListNode? h) { if (h?.next == null) return h;
    var newHead = ReverseRec(h.next); h.next.next = h; h.next = null; return newHead; }
```

Q59. Detect a cycle and find where it starts (Floyd).

```
ListNode? DetectCycle(ListNode? head) {
    ListNode? slow = head, fast = head;
    while (fast?.next != null) { slow = slow!.next; fast = fast.next.next;
        if (slow == fast) { slow = head; while (slow != fast) { slow = slow!.next; fast = fast!.next; }
    return slow; } }
    return null;
}
```

Q60. Merge two sorted lists.

```
ListNode? MergeTwo(ListNode? a, ListNode? b) { var dummy = new ListNode(0); var t = dummy;
    while (a != null && b != null) { if (a.val <= b.val) { t.next = a; a = a.next; } else { t.next = b; b
    = b.next; } t = t.next; }
    t.next = a ?? b; return dummy.next; }
```

> **Interview tip:** Dummy head node removes special cases for the first element.

Q61. Find the middle node.

Slow moves 1, fast moves 2; when fast ends, slow is the middle.

Q62. Remove the Nth node from the end.

Move fast n steps ahead, then move both until fast reaches the end.

```
ListNode? RemoveNthFromEnd(ListNode head, int n) { var dummy = new ListNode(0, head); ListNode fast =
    dummy, slow = dummy;
    for (int i = 0; i <= n; i++) fast = fast.next!;
    while (fast != null) { fast = fast.next!; slow = slow.next!; }
    slow.next = slow.next!.next; return dummy.next; }
```

Q63. Intersection of two linked lists.

Two pointers that switch to the other list's head at the end; they meet at the intersection (or both null) after at most m + n steps.

```
ListNode? GetIntersection(ListNode? a, ListNode? b) { ListNode? p = a, q = b;
    while (p != q) { p = p == null ? b : p.next; q = q == null ? a : q.next; } return p; }
```

Q64. Palindrome linked list in O(1) space.

Find middle, reverse the second half, compare halves, (optionally restore).

Q65. Add two numbers stored as reversed linked lists.

```
ListNode? AddTwoNumbers(ListNode? a, ListNode? b) { var d = new ListNode(0); var t = d; int carry = 0;
    while (a != null || b != null || carry > 0) { int s = (a?.val ?? 0) + (b?.val ?? 0) + carry;
        carry = s / 10; t.next = new ListNode(s % 10); t = t.next; a = a?.next; b = b?.next; }
    return d.next; }
```

Q66. Merge K sorted lists.

Min-heap of list heads. O(N log k).

```
ListNode? MergeKLists(ListNode?[] lists) {
    var pq = new PriorityQueue<ListNode, int>();
    foreach (var l in lists) if (l != null) pq.Enqueue(l, l.val);
    var d = new ListNode(0); var t = d;
    while (pq.TryDequeue(out var n, out _)) { t.next = n; t = n; if (n.next != null) pq.Enqueue(n.next, n.
    next.val); }
    return d.next;
}
```

Q67. Design an LRU Cache with O(1) get and put.

HashMap from key to node + doubly linked list ordered by recency. Full C# code is in the .NET guide (LinkedList + Dictionary). Explain: get moves node to front; put inserts at front and evicts the tail when full.

10. Stacks and Queues

Q68. Min Stack (getMin in O(1)).

Store pairs of (value, min so far).

```
public class MinStack { private readonly Stack<(int v, int min)> s = new();
    public void Push(int x) => s.Push((x, s.Count == 0 ? x : Math.Min(x, s.Peek().min)));
    public void Pop() => s.Pop(); public int Top() => s.Peek().v; public int GetMin() => s.Peek().min; }
```

Q69. Next Greater Element / Daily Temperatures (monotonic stack).

```
int[] DailyTemperatures(int[] t) { var res = new int[t.Length]; var st = new Stack<int>();
    for (int i = 0; i < t.Length; i++) {
        while (st.Count > 0 && t[st.Peek()] < t[i]) { int j = st.Pop(); res[j] = i - j; }
        st.Push(i); }
    return res; }
```

Q70. Evaluate Reverse Polish Notation.

```
int EvalRPN(string[] tokens) { var s = new Stack<int>();
    foreach (var t in tokens) {
        if (int.TryParse(t, out int n)) { s.Push(n); continue; }
        int b = s.Pop(), a = s.Pop();
        s.Push(t switch { "+" => a + b, "-" => a - b, "*" => a * b, _ => a / b });
    } return s.Pop(); }
```

Q71. Implement a queue using two stacks.

Push to `inStack`; on pop/peek, if `outStack` is empty move everything from `inStack`. Amortised O(1).

Q72. Largest Rectangle in Histogram.

Monotonic increasing stack of indices; when a lower bar arrives, pop and compute area with the popped height. O(n).

```
int LargestRectangle(int[] h) { var st = new Stack<int>(); int best = 0;
    for (int i = 0; i <= h.Length; i++) { int cur = i == h.Length ? 0 : h[i];
        while (st.Count > 0 && h[st.Peek()] >= cur) { int height = h[st.Pop()];
            int width = st.Count == 0 ? i : i - st.Peek() - 1; best = Math.Max(best, height * width); }
        st.Push(i); }
    return best; }
```

11. Recursion and Backtracking

Q73. Generate all subsets (power set).

```
IList<IList<int>> Subsets(int[] nums) { var res = new List<IList<int>>();
    void Bt(int i, List<int> cur) { if (i == nums.Length) { res.Add(new List<int>(cur)); return; }
        cur.Add(nums[i]); Bt(i + 1, cur); cur.RemoveAt(cur.Count - 1); Bt(i + 1, cur); }
    Bt(0, new()); return res; }
```

O(2^n * n).

Q74. Generate all permutations.

```
IList<IList<int>> Permute(int[] nums) { var res = new List<IList<int>>(); var used = new bool[nums.
    Length];
    void Bt(List<int> cur) { if (cur.Count == nums.Length) { res.Add(new List<int>(cur)); return; }
        for (int i = 0; i < nums.Length; i++) { if (used[i]) continue;
            used[i] = true; cur.Add(nums[i]); Bt(cur); cur.RemoveAt(cur.Count - 1); used[i] = false; } }
    Bt(new()); return res; }
```

Q75. Combination Sum (reuse allowed).

```
IList<IList<int>> CombinationSum(int[] c, int target) { var res = new List<IList<int>>();
    void Bt(int start, int rem, List<int> cur) {
        if (rem == 0) { res.Add(new List<int>(cur)); return; }
        for (int i = start; i < c.Length; i++) { if (c[i] > rem) continue;
            cur.Add(c[i]); Bt(i, rem - c[i], cur); cur.RemoveAt(cur.Count - 1); } }
    Bt(0, target, new()); return res; }
```

> **Interview tip:** Real-world: find which invoices combine to an unexplained payment amount (subset sum).

Q76. Generate Parentheses.

Add "(" while open < n, add ")" while close < open.

```
IList<string> GenerateParenthesis(int n) { var res = new List<string>();
    void Bt(string s, int open, int close) { if (s.Length == 2 * n) { res.Add(s); return; }
        if (open < n) Bt(s + "(", open + 1, close); if (close < open) Bt(s + ")", open, close + 1); }
    Bt("", 0, 0); return res; }
```

Q77. Word Search in a grid.

DFS from each cell, mark visited temporarily, backtrack. O(m n 4^L).

```
bool Exist(char[][] b, string w) {
    bool Dfs(int r, int c, int i) {
        if (i == w.Length) return true;
        if (r < 0 || c < 0 || r >= b.Length || c >= b[0].Length || b[r][c] != w[i]) return false;
        var tmp = b[r][c]; b[r][c] = '#';
        bool ok = Dfs(r + 1, c, i + 1) || Dfs(r - 1, c, i + 1) || Dfs(r, c + 1, i + 1) || Dfs(r, c - 1, i
    + 1);
        b[r][c] = tmp; return ok; }
    for (int r = 0; r < b.Length; r++) for (int c = 0; c < b[0].Length; c++) if (Dfs(r, c, 0)) return
    true;
    return false; }
```

Q78. N-Queens.

Place a queen per row; track used columns and both diagonals (`r - c` and `r + c`) in sets; backtrack. O(n!).

12. Trees and Binary Search Trees

Q79. Tree traversals: inorder, preorder, postorder, level order.

```
public class TreeNode(int val, TreeNode? left = null, TreeNode? right = null) {
    public int val = val; public TreeNode? left = left, right = right; }

void Inorder(TreeNode? n, List<int> r) { if (n == null) return; Inorder(n.left, r); r.Add(n.val); Inorder(
    n.right, r); }

IList<int> InorderIterative(TreeNode? root) { var res = new List<int>(); var st = new Stack<TreeNode>();
    var cur = root;
    while (cur != null || st.Count > 0) { while (cur != null) { st.Push(cur); cur = cur.left; }
        cur = st.Pop(); res.Add(cur.val); cur = cur.right; } return res; }

IList<IList<int>> LevelOrder(TreeNode? root) { var res = new List<IList<int>>(); if (root == null) return
    res;
    var q = new Queue<TreeNode>(); q.Enqueue(root);
    while (q.Count > 0) { var level = new List<int>();
        for (int n = q.Count; n > 0; n--) { var x = q.Dequeue(); level.Add(x.val);
            if (x.left != null) q.Enqueue(x.left); if (x.right != null) q.Enqueue(x.right); }
        res.Add(level); }
    return res; }
```

Q80. Maximum depth, and check if a tree is balanced.

```
int MaxDepth(TreeNode? n) => n == null ? 0 : 1 + Math.Max(MaxDepth(n.left), MaxDepth(n.right));
bool IsBalanced(TreeNode? root) { int H(TreeNode? n) { if (n == null) return 0;
        int l = H(n.left), r = H(n.right); if (l < 0 || r < 0 || Math.Abs(l - r) > 1) return -1; return 1
    + Math.Max(l, r); }
    return H(root) >= 0; }
```

Q81. Invert a binary tree / check same tree / symmetric tree.

```
TreeNode? Invert(TreeNode? n) { if (n == null) return null; (n.left, n.right) = (Invert(n.right), Invert(
    n.left)); return n; }
bool Same(TreeNode? a, TreeNode? b) => a == null || b == null ? a == b : a.val == b.val && Same(a.left, b.
    left) && Same(a.right, b.right);
bool Mirror(TreeNode? a, TreeNode? b) => a == null || b == null ? a == b : a.val == b.val && Mirror(a.
    left, b.right) && Mirror(a.right, b.left);
```

Q82. Diameter of a binary tree.

Longest path through each node = left height + right height; track the max while computing heights.

```
int Diameter(TreeNode? root) { int best = 0;
    int H(TreeNode? n) { if (n == null) return 0; int l = H(n.left), r = H(n.right); best = Math.Max(best,
     l + r); return 1 + Math.Max(l, r); }
    H(root); return best; }
```

Q83. Validate a Binary Search Tree.

Pass allowed (min, max) bounds down; checking only immediate children is the common mistake.

```
bool IsValidBST(TreeNode? n, long lo = long.MinValue, long hi = long.MaxValue) =>
    n == null || (n.val > lo && n.val < hi && IsValidBST(n.left, lo, n.val) && IsValidBST(n.right, n.val,
    hi));
```

Q84. Lowest Common Ancestor (binary tree and BST).

```
TreeNode? Lca(TreeNode? root, TreeNode p, TreeNode q) { if (root == null || root == p || root == q)
    return root;
    var l = Lca(root.left, p, q); var r = Lca(root.right, p, q); return l != null && r != null ? root : l
    ?? r; }

TreeNode LcaBst(TreeNode root, TreeNode p, TreeNode q) {
    while (true) { if (p.val < root.val && q.val < root.val) root = root.left!;
        else if (p.val > root.val && q.val > root.val) root = root.right!; else return root; } }
```

Q85. Kth smallest element in a BST.

Inorder traversal of a BST is sorted; stop at the kth visited node. O(h + k).

Q86. Binary Tree Right Side View.

Level-order traversal, take the last node of each level (or DFS visiting right first, recording the first node at each depth).

Q87. Serialize and deserialize a binary tree.

Preorder with a null marker.

```
string Serialize(TreeNode? n) => n == null ? "#" : $"{n.val},{Serialize(n.left)},{Serialize(n.right)}";
TreeNode? Deserialize(string data) { var q = new Queue<string>(data.Split(','));
    TreeNode? Build() { var v = q.Dequeue(); if (v == "#") return null;
        var n = new TreeNode(int.Parse(v)); n.left = Build(); n.right = Build(); return n; }
    return Build(); }
```

Q88. Binary Tree Maximum Path Sum.

At each node, gain = node + max(0, leftGain, rightGain); update answer with node + max(0,left) + max(0,right).

```
int MaxPathSum(TreeNode root) { int best = int.MinValue;
    int Gain(TreeNode? n) { if (n == null) return 0; int l = Math.Max(0, Gain(n.left)), r = Math.Max(0,
    Gain(n.right));
        best = Math.Max(best, n.val + l + r); return n.val + Math.Max(l, r); }
    Gain(root); return best; }
```

Q89. Construct a tree from preorder and inorder traversals.

The first preorder element is the root; its index in inorder splits left and right subtrees. Use a dictionary for O(1) index lookup. O(n).

Q90. BST insert, search and delete; complexities.

Search/insert/delete are O(h): O(log n) when balanced, O(n) when skewed. Delete cases: leaf (remove), one child (replace with child), two children (replace value with inorder successor then delete successor). Self-balancing trees (AVL, Red-Black - used by `SortedDictionary`) keep h = O(log n).

13. Heaps and Priority Queues

Q91. What is a heap? How does .NET PriorityQueue work?

A complete binary tree stored in an array where each parent is <= its children (min-heap). Insert and remove-min are O(log n), peek O(1), build O(n). `PriorityQueue<TElement, TPriority>` (.NET 6) is a min-heap; for a max-heap use a reversed comparer: `new PriorityQueue<int, int>(Comparer<int>.Create((a, b) => b.CompareTo(a)))`.

Q92. K Closest Points to Origin.

Max-heap of size k keyed by distance, or min-heap of all and take k.

```
int[][] KClosest(int[][] pts, int k) {
    var pq = new PriorityQueue<int[], int>(Comparer<int>.Create((a, b) => b.CompareTo(a)));
    foreach (var p in pts) { pq.Enqueue(p, p[0] * p[0] + p[1] * p[1]); if (pq.Count > k) pq.Dequeue(); }
    return pq.UnorderedItems.Select(x => x.Element).ToArray(); }
```

Q93. Find Median from Data Stream.

Max-heap for the lower half, min-heap for the upper half; keep sizes balanced. Add O(log n), median O(1).

```
public class MedianFinder {
    private readonly PriorityQueue<int, int> low = new(Comparer<int>.Create((a, b) => b.CompareTo(a)));
    private readonly PriorityQueue<int, int> high = new();
    public void AddNum(int x) { low.Enqueue(x, x); var m = low.Dequeue(); high.Enqueue(m, m);
        if (high.Count > low.Count) { var h = high.Dequeue(); low.Enqueue(h, h); } }
    public double FindMedian() => low.Count > high.Count ? low.Peek() : (low.Peek() + (double)high.Peek(
    )) / 2; }
```

Q94. Task Scheduler (CPU tasks with cooldown n).

Formula: `max(tasks.Length, (maxFreq - 1) * (n + 1) + countOfTasksWithMaxFreq)`. Or simulate with a max-heap and a cooldown queue.

14. Graphs

Q95. How do you represent a graph? BFS vs DFS?

Adjacency list (`Dictionary<int, List<int>>`) for sparse graphs (O(V + E) space); adjacency matrix for dense graphs (O(V squared)). BFS uses a queue, explores level by level, finds shortest paths in unweighted graphs. DFS uses recursion/stack, good for connectivity, cycle detection, topological sort. Both O(V + E).

Q96. Number of Islands.

```
int NumIslands(char[][] g) { int count = 0;
    void Dfs(int r, int c) { if (r < 0 || c < 0 || r >= g.Length || c >= g[0].Length || g[r][c] != '1')
    return;
        g[r][c] = '0'; Dfs(r + 1, c); Dfs(r - 1, c); Dfs(r, c + 1); Dfs(r, c - 1); }
    for (int r = 0; r < g.Length; r++) for (int c = 0; c < g[0].Length; c++) if (g[r][c] == '1') {
    count++; Dfs(r, c); }
    return count; }
```

Q97. Rotting Oranges / shortest path in a grid (multi-source BFS).

Push all rotten oranges into the queue at time 0, BFS level by level, count minutes; if any fresh remain, return -1.

```
int OrangesRotting(int[][] g) { var q = new Queue<(int, int)>(); int fresh = 0, mins = 0;
    for (int r = 0; r < g.Length; r++) for (int c = 0; c < g[0].Length; c++) { if (g[r][c] == 2) q.
    Enqueue((r, c)); else if (g[r][c] == 1) fresh++; }
    int[][] dirs = [[1, 0], [-1, 0], [0, 1], [0, -1]];
    while (q.Count > 0 && fresh > 0) { mins++;
        for (int n = q.Count; n > 0; n--) { var (r, c) = q.Dequeue();
            foreach (var d in dirs) { int nr = r + d[0], nc = c + d[1];
                if (nr < 0 || nc < 0 || nr >= g.Length || nc >= g[0].Length || g[nr][nc] != 1) continue;
                g[nr][nc] = 2; fresh--; q.Enqueue((nr, nc)); } } }
    return fresh == 0 ? mins : -1; }
```

Q98. Clone Graph.

DFS/BFS with a dictionary from original node to its clone to handle cycles.

Q99. Course Schedule: detect a cycle / topological sort (Kahn's algorithm).

```
int[] FindOrder(int n, int[][] prereq) {
    var adj = Enumerable.Range(0, n).Select(_ => new List<int>()).ToArray(); var indeg = new int[n];
    foreach (var p in prereq) { adj[p[1]].Add(p[0]); indeg[p[0]]++; }
    var q = new Queue<int>(Enumerable.Range(0, n).Where(i => indeg[i] == 0)); var order = new List<int>();
    while (q.Count > 0) { var u = q.Dequeue(); order.Add(u);
        foreach (var v in adj[u]) if (--indeg[v] == 0) q.Enqueue(v); }
    return order.Count == n ? order.ToArray() : [];   // empty = cycle exists
}
```

> **Interview tip:** Real-world: job dependency ordering (load files -> stage -> recon -> report), build systems, migration ordering.

Q100. Dijkstra's shortest path.

Min-heap of (distance, node); skip stale entries. O((V + E) log V). Doesn't work with negative weights (use Bellman-Ford).

```
int[] Dijkstra(List<(int to, int w)>[] adj, int src) {
    var dist = Enumerable.Repeat(int.MaxValue, adj.Length).ToArray(); dist[src] = 0;
    var pq = new PriorityQueue<int, int>(); pq.Enqueue(src, 0);
    while (pq.TryDequeue(out int u, out int d)) {
        if (d > dist[u]) continue;
        foreach (var (v, w) in adj[u]) if (d + w < dist[v]) { dist[v] = d + w; pq.Enqueue(v, dist[v]); }
    }
    return dist;
}
```

Q101. Union-Find (Disjoint Set Union) with path compression.

```
public class DSU { private readonly int[] p, rank;
    public DSU(int n) { p = Enumerable.Range(0, n).ToArray(); rank = new int[n]; }
    public int Find(int x) => p[x] == x ? x : p[x] = Find(p[x]);
    public bool Union(int a, int b) { a = Find(a); b = Find(b); if (a == b) return false;
        if (rank[a] < rank[b]) (a, b) = (b, a); p[b] = a; if (rank[a] == rank[b]) rank[a]++; return true;
    } }
```

Near O(1) per operation. Uses: number of connected components, redundant connection (cycle in undirected graph), Kruskal's MST, grouping accounts/merchants that share identifiers.

Q102. Word Ladder (shortest transformation sequence).

BFS over words; neighbours differ by one letter. Generate neighbours by trying 26 letters at each position against a HashSet. O(N * L * 26).

Q103. Minimum Spanning Tree: Kruskal vs Prim.

Kruskal: sort edges by weight, add if it doesn't form a cycle (DSU). O(E log E). Prim: grow from a node using a min-heap of edges. O(E log V). Use case: cheapest network to connect all branches.

Q104. Detect a cycle in a directed graph with DFS.

Three colours: white (unvisited), grey (on current path), black (done). Reaching a grey node means a back edge, i.e. a cycle.

15. Dynamic Programming

Q105. What is dynamic programming? Memoisation vs tabulation.

DP solves problems with **overlapping subproblems** and **optimal substructure** by storing results. Memoisation: top-down recursion + cache. Tabulation: bottom-up table, no recursion. Steps: define state, recurrence, base cases, order of computation, then optimise space.

Q106. Climbing Stairs (1 or 2 steps).

`ways(n) = ways(n-1) + ways(n-2)`; same as Fibonacci. O(n), O(1) with two variables.

Q107. House Robber.

```
int Rob(int[] a) { int prev = 0, cur = 0;
    foreach (var x in a) (prev, cur) = (cur, Math.Max(cur, prev + x)); return cur; }
```

Circular version: max of robbing houses 0..n-2 and 1..n-1.

Q108. Coin Change (minimum coins).

```
int CoinChange(int[] coins, int amount) { var dp = new int[amount + 1]; Array.Fill(dp, amount + 1); dp[0]
    = 0;
    for (int a = 1; a <= amount; a++) foreach (var c in coins) if (c <= a) dp[a] = Math.Min(dp[a], dp[a -
    c] + 1);
    return dp[amount] > amount ? -1 : dp[amount]; }
```

O(amount * coins). Number of ways variant: loop coins outside, amounts inside, `dp[a] += dp[a - c]`.

Q109. Longest Increasing Subsequence.

O(n squared) DP, or O(n log n) with patience sorting (binary search on tails).

```
int LengthOfLIS(int[] nums) { var tails = new List<int>();
    foreach (var x in nums) { int i = tails.BinarySearch(x); if (i < 0) i = ~i;
        if (i == tails.Count) tails.Add(x); else tails[i] = x; }
    return tails.Count; }
```

Q110. Longest Common Subsequence.

```
int LCS(string a, string b) { var dp = new int[a.Length + 1, b.Length + 1];
    for (int i = 1; i <= a.Length; i++) for (int j = 1; j <= b.Length; j++)
        dp[i, j] = a[i - 1] == b[j - 1] ? dp[i - 1, j - 1] + 1 : Math.Max(dp[i - 1, j], dp[i, j - 1]);
    return dp[a.Length, b.Length]; }
```

Used by diff tools.

Q111. Edit Distance (Levenshtein).

`dp[i,j]` = min edits to turn a[0..i) into b[0..j). If chars equal take `dp[i-1,j-1]`, else 1 + min(insert, delete, replace).

```
int MinDistance(string a, string b) { var dp = new int[a.Length + 1, b.Length + 1];
    for (int i = 0; i <= a.Length; i++) dp[i, 0] = i; for (int j = 0; j <= b.Length; j++) dp[0, j] = j;
    for (int i = 1; i <= a.Length; i++) for (int j = 1; j <= b.Length; j++)
        dp[i, j] = a[i - 1] == b[j - 1] ? dp[i - 1, j - 1] : 1 + Math.Min(dp[i - 1, j - 1], Math.Min(dp[i
    - 1, j], dp[i, j - 1]));
    return dp[a.Length, b.Length]; }
```

> **Interview tip:** Real-world: fuzzy matching merchant names or narrations in reconciliation ("AMAZON PAY" vs "AMAZONPAY IN").

Q112. 0/1 Knapsack.

```
int Knapsack(int[] wt, int[] val, int W) { var dp = new int[W + 1];
    for (int i = 0; i < wt.Length; i++) for (int w = W; w >= wt[i]; w--) dp[w] = Math.Max(dp[w], dp[w -
    wt[i]] + val[i]);
    return dp[W]; }
```

Iterate capacity backwards so each item is used once. Variants: subset sum, partition equal subset sum, target sum.

Q113. Partition Equal Subset Sum.

If total is odd, false; else knapsack-style boolean DP for total / 2.

```
bool CanPartition(int[] nums) { int s = nums.Sum(); if (s % 2 == 1) return false; var dp = new bool[s / 2
    + 1]; dp[0] = true;
    foreach (var x in nums) for (int t = s / 2; t >= x; t--) dp[t] |= dp[t - x]; return dp[s / 2]; }
```

Q114. Unique Paths in a grid.

`dp[r][c] = dp[r-1][c] + dp[r][c-1]`, 1D array optimisation. With obstacles, set blocked cells to 0.

Q115. Word Break.

```
bool WordBreak(string s, IList<string> dict) { var set = new HashSet<string>(dict); var dp = new bool[s.
    Length + 1]; dp[0] = true;
    for (int i = 1; i <= s.Length; i++) for (int j = 0; j < i && !dp[i]; j++) dp[i] = dp[j] && set.
    Contains(s[j..i]);
    return dp[s.Length]; }
```

Q116. Decode Ways ("12" -> "AB" or "L").

`dp[i] = (s[i-1] != '0' ? dp[i-1] : 0) + (10 <= two-digit <= 26 ? dp[i-2] : 0)`.

Q117. Maximum Product Subarray.

Track both max and min ending here (a negative number flips them).

```
int MaxProduct(int[] a) { int hi = a[0], lo = a[0], best = a[0];
    for (int i = 1; i < a.Length; i++) { if (a[i] < 0) (hi, lo) = (lo, hi);
        hi = Math.Max(a[i], hi * a[i]); lo = Math.Min(a[i], lo * a[i]); best = Math.Max(best, hi); }
    return best; }
```

16. Greedy and Intervals

Q118. Merge Intervals.

```
int[][] MergeIntervals(int[][] iv) { var s = iv.OrderBy(x => x[0]).ToList(); var res = new List<int[]>();
    foreach (var cur in s) { if (res.Count > 0 && res[^1][1] >= cur[0]) res[^1][1] = Math.Max(res[^1][1],
    cur[1]); else res.Add(cur); }
    return res.ToArray(); }
```

> **Interview tip:** Real-world: merging overlapping downtime windows or settlement cycles.

Q119. Insert Interval / Non-overlapping Intervals.

Insert: add all intervals ending before the new one, merge overlapping ones, add the rest. Non-overlapping (min removals): sort by end, greedily keep the interval that ends earliest, count conflicts.

Q120. Meeting Rooms II (minimum rooms needed).

Sort starts and ends separately and sweep, or use a min-heap of end times. O(n log n).

```
int MinMeetingRooms(int[][] iv) { var st = iv.Select(x => x[0]).Order().ToArray(); var en = iv.Select(x
    => x[1]).Order().ToArray();
    int rooms = 0, best = 0, j = 0;
    for (int i = 0; i < st.Length; i++) { if (st[i] < en[j]) rooms++; else j++; best = Math.Max(best,
    rooms); }
    return best; }
```

Q121. Jump Game.

Track the furthest reachable index; if the current index exceeds it, return false.

```
bool CanJump(int[] a) { int far = 0; for (int i = 0; i < a.Length; i++) { if (i > far) return false; far
    = Math.Max(far, i + a[i]); } return true; }
```

Q122. Gas Station.

If total gas < total cost, impossible. Otherwise reset the start whenever the running tank goes negative.

Q123. When does greedy fail? Give an example.

Coin change with coins {1, 3, 4} and amount 6: greedy picks 4+1+1 (3 coins), optimal is 3+3 (2 coins). Greedy works only when a local optimal choice is provably part of a global optimum (exchange argument).

17. Tries and Bit Manipulation

Q124. Implement a Trie (prefix tree).

```
public class Trie { private class Node { public Node?[] Next = new Node?[26]; public bool End; }
    private readonly Node root = new();
    public void Insert(string w) { var n = root; foreach (var c in w) n = n.Next[c - 'a'] ??= new Node();
    n.End = true; }
    private Node? Walk(string s) { var n = root; foreach (var c in s) { n = n.Next[c - 'a']; if (n ==
    null) return null; } return n; }
    public bool Search(string w) => Walk(w)?.End == true;
    public bool StartsWith(string p) => Walk(p) != null; }
```

Use cases: autocomplete, spell check, IP routing, BIN (card prefix) lookup.

Q125. Common bit manipulation tricks.

- Check odd: `(n & 1) == 1`.
- Power of two: `n > 0 && (n & (n - 1)) == 0`.
- Count set bits: `BitOperations.PopCount((uint)n)` or loop `n &= n - 1`.
- Single number (others appear twice): XOR everything.
- Swap without temp: `a ^= b; b ^= a; a ^= b;`
- Get/set/clear bit i: `(n >> i) & 1`, `n | (1 << i)`, `n & ~(1 << i)`.

> **Interview tip:** ISO 8583 uses a bitmap to indicate which data elements are present: bit i set means field i exists. Great example to mention.

Q126. Reverse bits / count bits for 0..n.

`dp[i] = dp[i >> 1] + (i & 1)` gives bit counts for all numbers in O(n).

18. System-Flavoured DSA and Real-World Scenarios

Q127. Reconcile two files of 10 million transactions each. Which data structures?

Load the smaller side into a `Dictionary<string, Txn>` keyed by a composite key (RRN + STAN + amount + date). Stream the larger file and probe: O(n + m) time. Matched: remove from dictionary; leftovers on either side are unmatched. If memory is not enough: external sort both files by key and do a merge-join (two pointers) in O(n log n), or push to SQL staging tables and use indexed joins. For tolerance matching (amount within 1 rupee, date +/- 1 day), sort by key and use a sliding window.

Q128. Design a rate limiter. Which algorithm and data structure?

Token bucket (tokens refill at a rate, request consumes one) or sliding window log (queue of timestamps per user; drop older than the window). Use a `Dictionary<user, Queue<long>>` in memory or Redis sorted sets for distributed limits.

Q129. Find the top 10 merchants by transaction count from a live stream.

Hash map of counts + min-heap of size 10 (O(log 10) per update). For huge streams with limited memory, approximate with Count-Min Sketch + heap.

Q130. Detect duplicate transactions within 5 minutes.

Sliding window: dictionary from key (card + amount + merchant) to the last timestamp, or a queue that evicts entries older than 5 minutes. O(1) per transaction.

Q131. How would you find the first unique transaction ID in a stream?

Dictionary for counts + a queue (or LinkedHashMap-style ordered structure) of candidates; pop from the front while its count > 1.

Q132. Implement autocomplete for merchant search.

Trie where each node stores top-k suggestions (precomputed by frequency), so a prefix query is O(prefix length). Alternatively a sorted array + binary search for the prefix range.

Q133. Which data structure for an undo/redo feature?

Two stacks: perform pushes onto undo and clears redo; undo pops from undo onto redo; redo does the reverse.

Q134. Which data structure for a job scheduler with priorities and delays?

Priority queue keyed by (run time, priority). A worker peeks the earliest job and sleeps until it's due. Persist to DB for durability.

19. Quick-Fire Questions

Q135. Array vs Linked List?

Array: O(1) random access, cache friendly, costly insert/delete in middle. Linked list: O(1) insert/delete at a known node, O(n) access, extra pointer memory.

Q136. Stack vs Queue?

Stack is LIFO (call stack, undo, DFS, parentheses). Queue is FIFO (BFS, task processing, buffering).

Q137. What is a hash collision and how is it handled?

Two keys map to the same bucket. Handled by chaining (list per bucket; .NET Dictionary uses chaining via entry indices) or open addressing (probing). A good hash function and load factor keep operations O(1) average; worst case O(n).

Q138. What is a balanced tree and why does it matter?

Height stays O(log n) (AVL, Red-Black), guaranteeing O(log n) search/insert/delete. Databases use B-trees/B+ trees (wide nodes, fewer disk reads) for indexes.

Q139. Why are SQL Server indexes B+ trees and not hash tables?

B+ trees support range queries, ordering and prefix searches (`BETWEEN`, `ORDER BY`, `LIKE 'abc%'`), stay balanced, and have high fan-out to minimise disk I/O. Hash indexes only support equality.

Q140. What is the difference between a tree and a graph?

A tree is a connected acyclic graph with n - 1 edges and a single path between any two nodes. Graphs can have cycles, be disconnected, directed or weighted.

Q141. Recursion vs iteration?

Recursion is clearer for trees/backtracking but uses stack space and risks overflow. Iteration with an explicit stack/queue avoids that. Tail-call optimisation is not guaranteed in C#.

Q142. What is a sentinel / dummy node?

An extra node at the head (or end) to remove special-case checks for empty lists or removing the first element.

Q143. What is memoisation in one line?

Caching a pure function's results by input so repeated calls are O(1).

Q144. What is the time complexity of common .NET collection operations?

`List.Add` O(1) amortised, `List.Insert(0)` O(n), `List.Contains` O(n), `Dictionary` get/set O(1) average, `HashSet.Contains` O(1), `SortedDictionary` O(log n), `PriorityQueue` enqueue/dequeue O(log n), `Array.Sort` O(n log n), `List.BinarySearch` O(log n) (on sorted data).

20. Arrays: More Interview Problems

Q145. Find the largest and second largest element in one pass.

Track `first` and `second`; if x > first, shift first to second; else if x > second and x != first, update second. O(n).

Q146. Reverse an array in place.

Two pointers swap from both ends until they meet. O(n), O(1).

Q147. Check if an array is sorted.

Loop i from 1, return false if a[i] < a[i-1]. O(n).

Q148. Remove an element (all occurrences) in place.

Write pointer copies elements not equal to val. Return new length. O(n).

Q149. Left rotate an array by one / by d.

By one: save a[0], shift left, put at end. By d: reversal algorithm (reverse 0..d-1, d..n-1, whole). O(n).

Q150. Find the leaders in an array (greater than all elements to their right).

Scan from right tracking max; element >= max so far is a leader. O(n).

Q151. Union and intersection of two sorted arrays.

Two pointers: advance smaller; equal values go to intersection (skip duplicates). O(m + n).

Q152. Find the maximum consecutive ones.

Counter reset on 0, track max. With at most k flips: sliding window counting zeros <= k.

Q153. Find the element that appears once when others appear twice.

XOR all elements. O(n), O(1).

Q154. Find the element that appears once when others appear three times.

Count bits at each of 32 positions modulo 3, or the `ones/twos` bitmask trick.

```
int SingleNumberIII(int[] a) { int ones = 0, twos = 0;
    foreach (var x in a) { ones = (ones ^ x) & ~twos; twos = (twos ^ x) & ~ones; } return ones; }
```

Q155. Find two elements that appear once (others twice).

XOR all to get x^y, isolate the lowest set bit `diff & -diff`, partition numbers by that bit and XOR each group.

Q156. Longest subarray with sum K (positives only).

Sliding window: expand right, shrink left while sum > K. O(n). With negatives: prefix sum + first-occurrence map.

Q157. Longest subarray with sum zero.

Prefix sum map storing first index of each sum; repeated sum means zero-sum subarray between indices.

Q158. Count subarrays with XOR equal to K.

Prefix XOR + hashmap counting `prefix ^ K` occurrences. O(n).

Q159. Rearrange array alternately positive and negative (equal counts).

Result array; positives at even indices, negatives at odd indices. O(n).

Q160. Next Permutation.

From right find first i with a[i] < a[i+1]; from right find first j with a[j] > a[i]; swap; reverse suffix after i.

```
void NextPermutation(int[] a) { int i = a.Length - 2; while (i >= 0 && a[i] >= a[i + 1]) i--;
    if (i >= 0) { int j = a.Length - 1; while (a[j] <= a[i]) j--; (a[i], a[j]) = (a[j], a[i]); }
    Array.Reverse(a, i + 1, a.Length - i - 1); }
```

Q161. Set Matrix Zeroes in O(1) extra space.

Use first row and first column as markers (plus a flag for column 0), then zero cells based on markers, then handle first row/column.

Q162. Pascal's Triangle.

Each row: `row[j] = prev[j-1] + prev[j]`, ends are 1. nCr for single element: multiply iteratively.

Q163. Majority Element II (appears more than n/3 times).

Extended Boyer-Moore with two candidates and counts, then verify counts. O(n), O(1).

Q164. 4Sum.

Sort, two nested loops fixing two numbers, two pointers for the rest, skip duplicates, use long for sum. O(n^3).

Q165. Merge Overlapping subarrays / find the missing and repeating number.

Missing and repeating in 1..n: use sum and sum of squares equations (with long) or XOR partition, or mark visited by negating values.

Q166. Count inversions in an array.

Modified merge sort: when taking from right half, add count of remaining elements in left half. O(n log n).

Q167. Reverse Pairs (i < j and a[i] > 2*a[j]).

Merge sort; before merging, count with a pointer over the right half for each left element. O(n log n).

Q168. Maximum product of three numbers.

Sort or track three largest and two smallest; answer = max(top3 product, top1 * smallest two).

Q169. Find all numbers disappeared in an array (1..n).

Mark index `abs(x)-1` negative; indices still positive are missing. O(n), O(1).

Q170. First missing positive (hard).

Place each x in 1..n at index x-1 by swapping (cyclic sort); first index i where a[i] != i+1 gives answer.

```
int FirstMissingPositive(int[] a) { int n = a.Length;
    for (int i = 0; i < n; i++) while (a[i] > 0 && a[i] <= n && a[a[i] - 1] != a[i]) { int j = a[i] - 1; (
    a[i], a[j]) = (a[j], a[i]); }
    for (int i = 0; i < n; i++) if (a[i] != i + 1) return i + 1; return n + 1; }
```

Compute the target index `j` before swapping so the index isn't recomputed from a value that is being changed.

Q171. Cyclic sort pattern - when to use?

When the array contains numbers in a known range 1..n: place each number at its correct index to find missing/duplicate numbers in O(n).

Q172. Best Time to Buy and Sell Stock with cooldown / fee.

State machine DP: hold, sold, rest. With fee: `cash = max(cash, hold + p - fee); hold = max(hold, cash - p)`.

Q173. Best Time to Buy and Sell Stock III (at most two transactions).

Track buy1, sell1, buy2, sell2 in one pass: `buy1 = max(buy1, -p); sell1 = max(sell1, buy1 + p); buy2 = max(buy2, sell1 - p); sell2 = max(sell2, buy2 + p)`.

Q174. Wiggle sort / alternate peaks.

For `a[0] <= a[1] >= a[2]...`: one pass swapping adjacent pairs that violate the pattern.

Q175. Candy distribution (ratings).

Two passes: left to right give more than left neighbour if higher rating; right to left ensure more than right neighbour. Sum. O(n).

Q176. Minimum number of platforms required at a railway station.

Sort arrivals and departures; sweep with two pointers counting platforms in use. O(n log n).

Q177. Chocolate distribution (minimise max-min among m students).

Sort, slide a window of size m, minimise `a[i+m-1] - a[i]`.

Q178. Kth smallest element in a sorted matrix.

Min-heap of first elements of rows, pop k times; or binary search on value range counting elements <= mid. O(n log(max-min)).

Q179. Search in a row-wise and column-wise sorted matrix.

Start at top-right: if greater than target move left, else move down. O(m + n).

Q180. Diagonal traverse / zigzag matrix.

Iterate by diagonal index d = r + c; alternate direction by parity.

Q181. Game of Life (in place).

Encode transitions in cell values (e.g. 2 = live to dead, 3 = dead to live), then normalise.

Q182. Subarray with maximum average of length k.

Fixed sliding window of sum; answer is max sum / k.

Q183. Shortest unsorted continuous subarray.

Left to right track max: last index where a[i] < max is the end; right to left track min: last index where a[i] > min is the start.

Q184. Find peak element.

Binary search: if a[mid] < a[mid+1], peak is on the right, else on the left. O(log n).

Q185. Range addition / difference array.

Add v to range [l, r]: `diff[l] += v; diff[r+1] -= v;` then prefix sum. O(1) per update.

21. Strings: More Interview Problems

Q186. Check if two strings are rotations of each other.

Same length and `(s1 + s1).Contains(s2)`. O(n) with KMP.

Q187. Remove duplicate characters keeping first occurrence.

HashSet of seen chars + StringBuilder. O(n).

Q188. Count vowels and consonants.

Loop with a vowel set; skip non-letters.

Q189. Reverse each word in a sentence.

Split by space, reverse each word's chars, join. Or in-place two-pointer reversal per word on char array.

Q190. Check if string has all unique characters.

HashSet or 26-length bool array / bitmask. O(n).

Q191. Find the first repeating character.

HashSet; first char that fails Add.

Q192. Check if string is a pangram.

26-bit mask or HashSet of lowercase letters; count == 26.

Q193. Isomorphic strings.

Two maps (s->t and t->s) ensuring consistent one-to-one mapping. O(n).

Q194. Word Pattern ("abba" vs "dog cat cat dog").

Same as isomorphic with words; check lengths match.

Q195. Valid Palindrome II (delete at most one char).

Two pointers; on mismatch, check if skipping left or right gives a palindrome. O(n).

Q196. Longest Palindrome that can be built from letters.

Sum of even counts plus (count-1) for odd counts, plus 1 if any odd.

Q197. Count palindromic substrings.

Expand around every centre counting expansions. O(n^2).

Q198. Roman to Integer and Integer to Roman.

Roman to int: add value, subtract if a smaller value precedes a larger one. Int to Roman: greedy with value table including 900, 400, 90, 40, 9, 4.

```
int RomanToInt(string s) { var v = new Dictionary<char, int> { ['I'] = 1, ['V'] = 5, ['X'] = 10, ['L'] =
    50, ['C'] = 100, ['D'] = 500, ['M'] = 1000 };
    int r = 0; for (int i = 0; i < s.Length; i++) r += (i + 1 < s.Length && v[s[i]] < v[s[i + 1]]) ? -v[s[
    i]] : v[s[i]]; return r; }
```

Q199. Add Binary / add two number strings.

Iterate from the end with carry; append digit; reverse at the end. O(max(m, n)).

Q200. Multiply Strings.

Array of size m + n; `pos[i+j+1] += d1*d2`, propagate carries. O(mn).

Q201. Count and Say.

Iteratively describe the previous term by run-length counting.

Q202. Zigzag Conversion.

Simulate rows with a direction flag, appending chars to row StringBuilders.

Q203. Text Justification (hard).

Greedily pack words per line; distribute spaces evenly with extra spaces to the left; last line left-justified.

Q204. Simplify Unix path.

Split by "/", use a stack: skip "" and ".", pop on "..", push names; join with "/".

Q205. Decode String ("3[a2[c]]" -> "accaccacc").

Two stacks (counts and strings) or recursion; on "]" pop and repeat.

```
string DecodeString(string s) { var counts = new Stack<int>(); var strs = new Stack<StringBuilder>(); var
    cur = new StringBuilder(); int k = 0;
    foreach (var c in s) {
        if (char.IsDigit(c)) k = k * 10 + (c - '0');
        else if (c == '[') { counts.Push(k); strs.Push(cur); cur = new StringBuilder(); k = 0; }
        else if (c == ']') { var rep = cur.ToString(); cur = strs.Pop(); for (int n = counts.Pop(); n > 0;
     n--) cur.Append(rep); }
        else cur.Append(c); }
    return cur.ToString(); }
```

Q206. Remove K digits to make the smallest number.

Monotonic increasing stack: pop while top > current and k > 0; strip leading zeros. O(n).

Q207. Remove duplicate letters (smallest lexicographic, each once).

Monotonic stack with last-occurrence index and in-stack set: pop larger chars that appear later.

Q208. Find all anagrams of p in s.

Fixed sliding window with counts; record start indices where counts match. O(n).

Q209. Longest substring with at most K distinct characters.

Sliding window with a count map; shrink when map size > K. O(n).

Q210. Minimum characters to add to make a palindrome (at front).

Compute LPS of `s + "#" + reverse(s)`; answer = n - lps[last].

Q211. Repeated Substring Pattern.

`(s + s).Substring(1, 2n - 2).Contains(s)`.

Q212. Longest Common Substring (not subsequence).

DP: `dp[i][j] = dp[i-1][j-1] + 1` if chars match, else 0; track max. O(mn).

Q213. Rabin-Karp: how does rolling hash work?

Hash of window updated in O(1): remove leading char contribution, multiply by base, add new char, modulo prime; verify on hash match to handle collisions.

Q214. Z-algorithm in brief.

Z[i] = length of longest substring starting at i matching the prefix; pattern search via `pattern + "$" + text`. O(n).

Q215. Check if a string is a valid number.

State machine or careful parsing: optional sign, digits, optional dot with digits, optional exponent with sign and digits.

Q216. Basic Calculator (+, -, parentheses).

Stack of (result, sign) pairs on "(", accumulate numbers with current sign. O(n).

Q217. Basic Calculator II (+, -, *, /).

Stack: push numbers with sign; for * and / apply to top immediately; sum stack at end.

Q218. Group shifted strings / encode-decode strings.

Shifted: key by differences between consecutive chars mod 26. Encode list of strings: length-prefix each (`"4#abcd"`), decode by reading length.

Q219. Longest word in a dictionary through deleting (subsequence check).

For each word check subsequence with two pointers; pick longest, then lexicographically smallest.

Q220. Is Subsequence.

Two pointers over s and t. O(n). For many queries, preprocess index lists per char and binary search.

22. Hashing, Two Pointers and Sliding Window (More)

Q221. Four Sum II (count tuples from four arrays summing to 0).

Map all sums of A+B with counts; for each C+D add count of -(c+d). O(n^2).

Q222. Contiguous Array (equal 0s and 1s).

Treat 0 as -1, prefix sum with first-index map; longest subarray with sum 0.

Q223. Subarray Sums Divisible by K.

Count prefix sums modulo K (normalise negatives) in a map. O(n).

Q224. Continuous Subarray Sum (multiple of k, length >= 2).

Map of prefix mod k to first index; if same mod seen at least two positions ago, true.

Q225. Happy Number.

Floyd cycle detection on the digit-square-sum sequence, or HashSet of seen numbers.

Q226. Isomorphic / Longest substring with same letters after replacement - recognising window problems.

Signals: "contiguous", "longest/shortest subarray/substring", "at most K". Use variable-size window with a validity condition.

Q227. Fruit Into Baskets.

Longest subarray with at most 2 distinct values: sliding window with counts.

Q228. Max Consecutive Ones III.

Window with zero count <= k; expand right, shrink left when zeros exceed k.

Q229. Minimum Size Subarray Sum >= target.

Sliding window (positives): expand, then shrink while sum >= target, record min length. O(n).

Q230. Number of subarrays with product less than K.

Window product; shrink while product >= k; add `r - l + 1` per right index.

Q231. Count number of nice subarrays (exactly k odd numbers).

atMost(k) - atMost(k - 1) trick with sliding window.

Q232. Subarrays with K different integers.

Same "exactly K = atMost(K) - atMost(K-1)" trick.

Q233. Squares of a sorted array.

Two pointers from both ends placing larger squares from the back. O(n).

Q234. Boats to save people.

Sort, two pointers: pair heaviest with lightest if within limit. Count boats.

Q235. Partition labels.

Last index of each char; extend current partition end to max last index; cut when i == end.

Q236. Valid Sudoku.

HashSets for rows, columns and 3x3 boxes (box index `(r/3)*3 + c/3`).

Q237. Design a phone directory / dedupe transactions by composite key.

Use HashSet of value tuples `(rrn, stan, amount)`; record structs give value equality and hashing.

Q238. Brick Wall (fewest crossed bricks).

Count edge positions (prefix sums per row, excluding the end) in a map; answer = rows - max count.

Q239. Rotate string / Bulls and Cows.

Bulls and Cows: count exact matches, then cows from min of character counts minus bulls.

Q240. Shortest word distance.

Single pass tracking last index of each word; update min difference.

23. Binary Search (More)

Q241. Square root of x (integer).

Binary search on [0, x] for largest m with m*m <= x (use long). O(log x).

Q242. Find the Nth root of M.

Binary search on [1, M] with power computed carefully to avoid overflow.

Q243. Single element in a sorted array (others twice).

Binary search on even indices: if a[mid] == a[mid+1] (mid even) answer is right, else left. O(log n).

Q244. Search insert position.

Lower bound: first index with a[i] >= target.

Q245. Floor and ceil in a sorted array.

Floor: last element <= x; ceil: first element >= x; binary search variants.

Q246. Count occurrences of an element in a sorted array.

upperBound - lowerBound.

Q247. Find the rotation count of a sorted rotated array.

Index of the minimum element via binary search.

Q248. Search in rotated sorted array with duplicates.

Same as before, but if a[lo] == a[mid] == a[hi], shrink both ends (worst case O(n)).

Q249. Aggressive cows (maximise minimum distance).

Sort stalls; binary search the distance; greedy check placing cows. O(n log range).

Q250. Book allocation / split array largest sum / painters partition.

Binary search the max pages per student between max(a) and sum(a); greedy count of students needed <= m.

Q251. Capacity to ship packages within D days.

Binary search capacity in [max weight, total]; greedy days count. O(n log sum).

Q252. Minimum days to make m bouquets.

Binary search the day; count bouquets of k adjacent bloomed flowers.

Q253. Kth missing positive number.

Binary search on index: missing count at i is `a[i] - (i + 1)`; answer = lo + k.

Q254. Median of a row-wise sorted matrix.

Binary search on value range; count elements <= mid per row with upper bound. O(r log c log range).

Q255. Find K closest elements.

Binary search the left bound of a window of size k: compare `x - a[mid]` with `a[mid+k] - x`. O(log(n-k) + k).

Q256. Time-based key-value store.

Dictionary of key to list of (timestamp, value) appended in increasing order; get with binary search for largest timestamp <= t.

Q257. Minimise maximum distance to gas station.

Binary search on answer (doubles) with precision, counting stations needed per gap.

Q258. Find a peak element in a 2D grid.

Binary search on columns: find max in mid column; move toward larger neighbour. O(n log m).

Q259. Why does binary search on the answer work?

The feasibility function is monotonic: if capacity C works, any larger works. So we search the boundary between infeasible and feasible.

Q260. Common binary search bugs?

Infinite loops when `lo = mid` with `mid = (lo+hi)/2` (use upper mid), off-by-one on bounds, overflow in mid calculation, wrong loop condition (`<` vs `<=`).

24. Linked Lists (More)

Q261. Insert a node at head, tail and a given position.

Head: new node points to head. Tail: traverse to last (or keep a tail pointer). Position: walk to node at pos-1 and relink. O(1)/O(n)/O(n).

Q262. Delete a node given only a pointer to it (not tail).

Copy the next node's value into it and skip the next node: `node.val = node.next.val; node.next = node.next.next;`.

Q263. Find the length of a linked list (iterative and recursive).

Count while traversing; recursive `1 + Len(next)`.

Q264. Search an element in a linked list.

Linear traversal. O(n).

Q265. Remove duplicates from a sorted list.

If current.val == next.val, skip next; else move. O(n).

Q266. Remove duplicates from an unsorted list.

HashSet of seen values; unlink repeats. O(n) time and space.

Q267. Remove all duplicates from a sorted list (keep only distinct).

Dummy node; when a run of duplicates is found, skip the entire run.

Q268. Remove linked list elements with a given value.

Dummy head; unlink nodes where next.val == val.

Q269. Odd Even Linked List (group odd and even positioned nodes).

Two pointers building odd and even chains; attach even head after odd tail. O(n), O(1).

Q270. Partition list around x (keeping order).

Two dummy lists for < x and >= x; join. O(n).

Q271. Rotate list by k places.

Get length, connect tail to head to form a ring, break at `len - k % len`.

Q272. Reverse a linked list between positions m and n.

Walk to node before m; repeatedly move the next node to the front of the sublist (head insertion) n - m times.

Q273. Reverse nodes in k-groups.

Check k nodes exist; reverse them; connect previous group tail to new head; recurse or iterate.

```
ListNode? ReverseKGroup(ListNode? head, int k) {
    var node = head; for (int i = 0; i < k; i++) { if (node == null) return head; node = node.next; }
    ListNode? prev = ReverseKGroup(node, k), cur = head;
    for (int i = 0; i < k; i++) { var nxt = cur!.next; cur.next = prev; prev = cur; cur = nxt; }
    return prev; }
```

Q274. Swap nodes in pairs.

Dummy node; for each pair relink `prev -> second -> first -> rest`.

Q275. Reorder list (L0 -> Ln -> L1 -> Ln-1 ...).

Find middle, reverse second half, merge alternately. O(n), O(1).

Q276. Copy List with Random Pointer.

Interleave copies (A -> A' -> B -> B'), set `copy.random = orig.random.next`, then separate. O(n), O(1) extra. Or dictionary mapping originals to copies.

Q277. Sort a linked list.

Merge sort: find middle with slow/fast, sort halves, merge. O(n log n), O(log n) stack.

Q278. Flatten a multilevel doubly linked list.

DFS: when a node has a child, splice the flattened child list between node and next.

Q279. Flatten a linked list where each node has a down list (sorted).

Merge lists pairwise from the right using merge of two sorted lists via down pointers.

Q280. Detect if linked list is circular (last points to head).

Traverse until null (not circular) or back at head (circular).

Q281. Delete the middle node of a linked list.

Slow/fast with a prev pointer; skip slow.

Q282. Add 1 to a number represented as a linked list.

Reverse, add with carry, reverse back; or recursion returning carry.

Q283. Implement a doubly linked list with add/remove in O(1).

Nodes with prev/next and sentinel head/tail nodes to avoid null checks; basis for LRU cache.

Q284. Why use a dummy node in linked list problems?

Removes special cases when the head changes (insert/delete at head) and simplifies building new lists.

Q285. When would you choose a linked list in a real system?

Rarely in .NET business code (List<T> is faster due to cache locality); useful for LRU caches, queues with O(1) removal of known nodes, and undo history.

25. Stacks and Queues (More)

Q286. Implement a stack using an array and a linked list.

Array: top index with resize. Linked list: push/pop at head. O(1) each.

Q287. Implement a stack using queues.

One queue: on push, enqueue then rotate the previous elements behind it (push O(n), pop O(1)).

Q288. Implement a circular queue.

Fixed array with head, tail and count; indices wrap with modulo.

```
public class CircularQueue(int k) { private readonly int[] a = new int[k]; private int head, count;
    public bool Enqueue(int v) { if (count == a.Length) return false; a[(head + count++) % a.Length] = v;
    return true; }
    public bool Dequeue() { if (count == 0) return false; head = (head + 1) % a.Length; count--; return
    true; }
    public int Front() => count == 0 ? -1 : a[head]; public int Rear() => count == 0 ? -1 : a[(head +
    count - 1) % a.Length]; }
```

Q289. Implement a deque.

Doubly linked list or circular array supporting add/remove at both ends.

Q290. Next Greater Element II (circular array).

Iterate 2n times over indices mod n with a monotonic stack.

Q291. Previous smaller element.

Monotonic increasing stack from left; top after popping larger-or-equal elements is the previous smaller.

Q292. Stock Span problem.

Monotonic stack of (price, span); pop while top price <= current, accumulating spans.

Q293. Asteroid Collision.

Stack: when current is negative and top positive, resolve collisions by size.

Q294. Remove adjacent duplicates in a string.

Stack (or StringBuilder as stack): pop if top equals current, else push.

Q295. Infix to postfix conversion.

Shunting-yard: output operands; push operators after popping higher/equal precedence ones; handle parentheses.

Q296. Evaluate an infix expression.

Two stacks (values and operators) with precedence handling, or convert to postfix then evaluate.

Q297. Maximal Rectangle in a binary matrix.

For each row build histogram heights and apply largest rectangle in histogram. O(rows * cols).

Q298. Sum of subarray minimums.

For each element, count subarrays where it is the minimum using previous-less and next-less-or-equal distances via monotonic stacks; sum `a[i] * left * right` mod 1e9+7.

Q299. Online stock span / design a browser history.

Browser history: two stacks (back and forward) or a list with a current index.

Q300. Sliding window minimum.

Monotonic increasing deque of indices (mirror of sliding window maximum).

Q301. First negative number in every window of size k.

Queue of indices of negative numbers; drop indices outside the window.

Q302. Reverse first K elements of a queue.

Pop k into a stack, enqueue them back, then move the remaining n-k from front to back.

Q303. Generate binary numbers 1 to n using a queue.

Start with "1"; dequeue s, output, enqueue s+"0" and s+"1".

Q304. Celebrity problem.

Stack elimination (or two pointers): if A knows B, A isn't a celebrity; then verify the remaining candidate. O(n).

Q305. Longest Valid Parentheses.

Stack of indices starting with -1; on ")" pop, if empty push index, else length = i - top. O(n).

Q306. Car Fleet.

Sort by position descending; compute time to target; stack counts fleets when a car's time exceeds the top.

Q307. Online Stock Span / Exclusive time of functions.

Exclusive time: stack of function ids; on start, add elapsed time to the current top; on end, add time including end and pop.

Q308. When to use a monotonic stack vs deque?

Stack: next/previous greater/smaller for each element. Deque: sliding window max/min where elements also expire from the front.

Q309. What is the time complexity of monotonic stack algorithms?

O(n): each element is pushed and popped at most once.

Q310. Implement a priority queue without PriorityQueue class.

Binary heap on a List: add at end and sift up; remove root by moving last to root and sift down.

26. Recursion and Backtracking (More)

Q311. Print 1 to N and N to 1 using recursion.

Recursive call before or after printing determines the order.

Q312. Calculate power x^n efficiently (fast exponentiation).

```
double MyPow(double x, long n) { if (n < 0) { x = 1 / x; n = -n; } double r = 1;
    while (n > 0) { if ((n & 1) == 1) r *= x; x *= x; n >>= 1; } return r; }
```

O(log n).

Q313. Tower of Hanoi.

Move n-1 to auxiliary, move largest to target, move n-1 onto it. 2^n - 1 moves.

Q314. Reverse a stack using recursion.

Pop top, reverse the rest, insert the popped item at the bottom (another recursive helper).

Q315. Sort a stack using recursion.

Pop, sort rest, insert popped element in sorted position recursively.

Q316. Subsets II (with duplicates).

Sort; in the loop skip `nums[i] == nums[i-1]` when i > start.

Q317. Permutations II (with duplicates).

Sort; skip if `used[i-1]` is false and `nums[i] == nums[i-1]`.

Q318. Combination Sum II (each number once, no duplicate combos).

Sort; recurse with `i + 1`; skip duplicates at the same level.

Q319. Combinations (n choose k).

Backtracking from start to n, prune when remaining numbers are fewer than needed.

Q320. Letter Combinations of a Phone Number.

Map digits to letters; DFS building strings. O(4^n).

Q321. Palindrome Partitioning.

At each start index, try every end where substring is a palindrome; recurse. Precompute palindrome table for speed.

Q322. Restore IP addresses.

Backtrack choosing 1-3 digit segments, 4 parts, each 0-255 without leading zeros.

Q323. Sudoku Solver.

Find an empty cell, try digits 1-9 valid by row/col/box sets, recurse, undo on failure.

Q324. N-Queens full code.

```
IList<IList<string>> SolveNQueens(int n) { var res = new List<IList<string>>(); var cols = new
    HashSet<int>(); var d1 = new HashSet<int>(); var d2 = new HashSet<int>();
    var board = Enumerable.Range(0, n).Select(_ => new string('.', n).ToCharArray()).ToArray();
    void Bt(int r) { if (r == n) { res.Add(board.Select(b => new string(b)).ToList()); return; }
        for (int c = 0; c < n; c++) { if (cols.Contains(c) || d1.Contains(r - c) || d2.Contains(r + c))
    continue;
            cols.Add(c); d1.Add(r - c); d2.Add(r + c); board[r][c] = 'Q'; Bt(r + 1);
            cols.Remove(c); d1.Remove(r - c); d2.Remove(r + c); board[r][c] = '.'; } }
    Bt(0); return res; }
```

Q325. Rat in a Maze (all paths).

DFS in four directions with visited marking, record path strings at destination.

Q326. Word Search II (many words).

Build a Trie of words and DFS the grid once, following trie nodes; remove found words to prune. Much faster than searching each word.

Q327. M-coloring problem.

Assign colours to vertices one by one, checking adjacent vertices; backtrack.

Q328. Partition to K equal sum subsets.

Sort descending, backtrack placing each number into a bucket with remaining capacity; skip identical bucket states.

Q329. Matchsticks to square.

Same as partition into 4 equal subsets.

Q330. Generate all binary strings without consecutive 1s.

Recurse appending 0 always and 1 only if last char isn't 1.

Q331. Count paths in a grid with recursion vs memoisation.

Plain recursion is exponential; memoise `(r, c)` to get O(mn).

Q332. Expression Add Operators.

Backtrack inserting +, -, * between digits, tracking value and last operand for multiplication precedence.

Q333. What is the difference between recursion and backtracking?

Backtracking is recursion that builds candidates incrementally and abandons (undoes) choices that can't lead to a valid solution.

Q334. How do you estimate backtracking complexity?

Branching factor ^ depth times work per node (e.g. permutations n!, subsets 2^n, combinations C(n,k)).

Q335. What is pruning?

Cutting branches early using constraints (sorted input with sum exceeding target, visited sets), drastically reducing search space.

27. Trees (More)

Q336. Count nodes, sum of nodes, count leaves.

Simple recursion: `Count(n) = n == null ? 0 : 1 + Count(left) + Count(right)`.

Q337. Preorder and postorder iterative traversal.

Preorder: stack, push right then left. Postorder: two stacks, or reverse of modified preorder (root, right, left).

Q338. Morris inorder traversal (O(1) space).

For each node, find inorder predecessor; create a temporary thread back to the node, then remove it on second visit.

Q339. Zigzag level order traversal.

BFS with a flag; reverse every other level (or insert at front).

Q340. Bottom view and top view of a binary tree.

BFS with horizontal distance; top view keeps first node per HD, bottom view keeps last. Sort by HD.

Q341. Left view of a binary tree.

First node of each level in BFS, or DFS visiting left first recording first node per depth.

Q342. Vertical order traversal.

BFS/DFS with (column, row); sort by column, then row, then value.

Q343. Boundary traversal.

Root, left boundary (excluding leaves), all leaves left to right, right boundary in reverse.

Q344. Check if two trees are identical vs subtree of another.

Identical: recursive compare. Subtree: for each node in big tree, check identical (O(mn)), or serialise and string match.

Q345. Path Sum (root to leaf equals target).

Recurse subtracting values; check at leaves.

Q346. Path Sum II (all paths) and Path Sum III (any downward path).

II: backtracking with a path list. III: prefix sums on root-to-node path with a count map. O(n).

Q347. Sum Root to Leaf Numbers.

DFS carrying `num * 10 + val`; add at leaves.

Q348. Maximum width of a binary tree.

BFS with positional index (left 2i, right 2i+1); width = last - first + 1 per level (use long/normalise).

Q349. Populate next right pointers.

BFS per level, or O(1) space using the previous level's next pointers.

Q350. Flatten binary tree to linked list (preorder).

Reverse postorder (right, left, root) with a prev pointer, or Morris-like iterative approach.

Q351. Convert sorted array to height-balanced BST.

Middle element as root, recurse on halves. O(n).

Q352. Convert sorted linked list to BST.

Inorder simulation with list pointer and size, O(n).

Q353. Insert into a BST.

Recurse left or right until null, create node.

Q354. Delete a node in a BST.

```
TreeNode? DeleteNode(TreeNode? root, int key) {
    if (root == null) return null;
    if (key < root.val) root.left = DeleteNode(root.left, key);
    else if (key > root.val) root.right = DeleteNode(root.right, key);
    else { if (root.left == null) return root.right; if (root.right == null) return root.left;
        var s = root.right; while (s.left != null) s = s.left; root.val = s.val; root.right = DeleteNode(
    root.right, s.val); }
    return root; }
```

Q355. Floor and ceil in a BST.

Walk down: if node value <= key, candidate floor and go right; else go left (ceil symmetric).

Q356. Inorder successor in a BST.

If right subtree exists, its leftmost; else track last ancestor where we went left.

Q357. Two Sum in a BST.

Inorder to sorted list + two pointers, or BST iterators from both ends. O(n).

Q358. BST iterator (next in O(1) amortised).

Stack holding the left path; next pops and pushes the left path of the right child.

Q359. Recover a BST where two nodes are swapped.

Inorder traversal; find the two out-of-order nodes (first and second violation), swap values.

Q360. Largest BST in a binary tree.

Postorder returning (isBST, size, min, max) per subtree.

Q361. Construct BST from preorder.

Recursive with upper bound: create node if value < bound, recurse left with node value, right with bound. O(n).

Q362. Construct binary tree from inorder and postorder.

Last postorder element is the root; split by its inorder index; build right subtree first when consuming postorder from the end.

Q363. Check if a tree is a complete binary tree.

BFS; after seeing the first null child, no non-null nodes may appear.

Q364. Check if a tree is a full binary tree.

Every node has 0 or 2 children.

Q365. Count nodes in a complete binary tree in less than O(n).

Compare leftmost and rightmost heights; if equal, 2^h - 1; else recurse. O(log^2 n).

Q366. Children sum property / convert tree to sum tree.

Postorder recursion updating node values from children sums.

Q367. Burn a binary tree from a target node (time to burn all).

Build parent pointers, BFS from target in three directions (left, right, parent).

Q368. All nodes at distance K from a target.

Parent map + BFS from target K levels.

Q369. Distance between two nodes.

`dist(root, a) + dist(root, b) - 2 * dist(root, lca)`.

Q370. Print all root-to-leaf paths.

DFS with a path list; record at leaves; backtrack.

Q371. Mirror a tree / check foldable tree.

Mirror: swap children recursively. Foldable: left subtree is a mirror structure of right subtree.

Q372. Convert binary tree to doubly linked list (inorder).

Inorder traversal with a prev pointer linking `prev.right = cur; cur.left = prev`.

Q373. Serialize with level order (LeetCode format).

BFS writing "null" for missing children; deserialise with a queue assigning children in order.

Q374. What is an AVL tree and rotations?

Self-balancing BST where heights of children differ by at most 1. Rebalance with left/right rotations (LL, RR, LR, RL cases). O(log n) operations.

Q375. What is a Red-Black tree?

Balanced BST with colour rules (root black, red nodes have black children, equal black height on all paths). Fewer rotations than AVL; used in SortedDictionary/SortedSet.

Q376. What is a B-tree / B+ tree?

Balanced multi-way trees with many keys per node; B+ trees store data in linked leaves, ideal for disk-based indexes and range scans (SQL Server indexes).

Q377. What is a segment tree?

Tree over array ranges supporting range queries (sum/min) and point updates in O(log n). Lazy propagation for range updates.

Q378. What is a Fenwick tree (Binary Indexed Tree)?

Array-based structure for prefix sums with O(log n) update and query; simpler than a segment tree.

```
void Update(int[] bit, int i, int delta) { for (i++; i < bit.Length; i += i & -i) bit[i] += delta; }
int Query(int[] bit, int i) { int s = 0; for (i++; i > 0; i -= i & -i) s += bit[i]; return s; }
```

Q379. Tree DP: House Robber III.

Return (rob, notRob) for each subtree: `rob = val + left.notRob + right.notRob`, `notRob = max(left) + max(right)`.

Q380. Binary Tree Cameras (minimum cameras).

Greedy postorder with states: needs camera, has camera, covered. Place cameras at parents of leaves.

Q381. Number of unique BSTs with n nodes.

Catalan number via DP: `dp[n] = sum(dp[i-1] * dp[n-i])`.

Q382. Height of a tree from parent array.

Compute depth of each node with memoisation following parents; max depth.

28. Heaps (More)

Q383. Heapify an array (build heap) - why O(n)?

Sift down from the last non-leaf to the root. Most nodes are near the bottom and move little, so total work sums to O(n).

Q384. Heap sort.

Build a max-heap, swap root with the last element, shrink heap, sift down. O(n log n), in place, not stable.

Q385. Kth smallest element using a max-heap.

Keep a max-heap of size k; root is the answer. O(n log k).

Q386. Sort a nearly sorted (k-sorted) array.

Min-heap of size k+1; pop minimum as you push the next element. O(n log k).

Q387. Merge K sorted arrays.

Min-heap of (value, array index, element index). O(N log k).

Q388. Smallest range covering elements from K lists.

Min-heap with current max tracked; pop min, push next from same list; update best range until any list ends.

Q389. Top K frequent words (ties lexicographic).

Count map, then sort or heap with comparer (frequency desc, word asc).

Q390. Reorganize String (no two adjacent same).

Max-heap by count; pop two different chars each step; if one char's count > (n+1)/2, impossible.

Q391. IPO / maximise capital.

Sort projects by capital; push affordable ones' profits into a max-heap; take k best.

Q392. Minimum cost to connect ropes.

Min-heap; repeatedly combine two smallest, add cost, push back. O(n log n).

Q393. Last stone weight.

Max-heap; smash two largest, push difference.

Q394. Kth largest element in a stream.

Min-heap of size k; add then pop if size > k; peek is answer.

Q395. Hand of Straights / divide into consecutive groups.

Sorted counts (SortedDictionary or min-heap); start groups from the smallest available card.

Q396. Single-threaded CPU scheduling.

Sort tasks by enqueue time; min-heap by (processing time, index); simulate time.

Q397. Why use a heap rather than sorting?

When you need repeated min/max with insertions (streams) or only top K of a large set: O(n log k) vs O(n log n).

29. Graphs (More)

Q398. BFS template for adjacency list.

```
var dist = new int[n]; Array.Fill(dist, -1); var q = new Queue<int>(); q.Enqueue(src); dist[src] = 0;
while (q.Count > 0) { var u = q.Dequeue(); foreach (var v in adj[u]) if (dist[v] == -1) { dist[v] = dist[
    u] + 1; q.Enqueue(v); } }
```

Q399. DFS template (recursive and iterative).

Recursive with visited array; iterative with explicit stack (mark visited when pushing or popping consistently).

Q400. Number of connected components in an undirected graph.

Count DFS/BFS starts from unvisited nodes, or Union-Find counting successful unions.

Q401. Detect a cycle in an undirected graph.

DFS tracking parent (visited neighbour that isn't the parent means a cycle) or Union-Find (union of already-connected nodes).

Q402. Is the graph bipartite?

BFS/DFS 2-colouring; conflict means not bipartite.

Q403. Flood Fill.

DFS/BFS from the start pixel recolouring cells with the original colour (guard if new colour equals old).

Q404. Surrounded Regions.

Mark 'O's connected to the border (DFS from border), flip all other 'O' to 'X', restore marked.

Q405. Pacific Atlantic Water Flow.

Reverse BFS from each ocean's border cells going uphill; answer is the intersection.

Q406. Number of enclaves / closed islands.

Remove land connected to the border, count the remaining land cells or islands.

Q407. Max area of island.

DFS returning area of each island; track max.

Q408. Number of distinct islands.

Record shape as the sequence of DFS moves (or relative coordinates) in a HashSet.

Q409. 01 Matrix (distance to nearest 0).

Multi-source BFS from all zeros.

Q410. Walls and gates.

Multi-source BFS from all gates.

Q411. Shortest path in a binary matrix (8 directions).

BFS from (0,0) counting path length. O(n^2).

Q412. Snakes and Ladders.

BFS over board squares with dice moves 1-6, applying ladders/snakes.

Q413. Open the Lock.

BFS over 4-digit states with deadends set; bidirectional BFS speeds it up.

Q414. Topological sort using DFS.

Postorder push to a stack; reverse gives order; detect cycles with colours.

Q415. Alien Dictionary.

Build edges from first differing chars of adjacent words; topological sort; invalid if a word is followed by its prefix.

Q416. Course Schedule II / parallel courses (minimum semesters).

Kahn's algorithm level by level; number of levels is the minimum semesters.

Q417. Network Delay Time.

Dijkstra from source; answer is max distance or -1 if unreachable.

Q418. Cheapest Flights Within K Stops.

Bellman-Ford limited to K+1 iterations (copy distances each round) or BFS with stops. O(K * E).

Q419. Path with minimum effort.

Dijkstra where path cost is the max edge difference, or binary search + BFS.

Q420. Swim in Rising Water.

Dijkstra/priority queue minimising the maximum elevation on the path.

Q421. Bellman-Ford algorithm.

Relax all edges V-1 times; a further improvement means a negative cycle. O(VE). Handles negative weights.

Q422. Floyd-Warshall algorithm.

All-pairs shortest paths: `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])` for each k. O(V^3).

Q423. 0-1 BFS.

Edges with weight 0 or 1: deque, push front for 0-weight, back for 1-weight. O(V + E).

Q424. Prim's algorithm implementation idea.

Start at any node, min-heap of edges from visited set, add the cheapest edge to an unvisited node.

Q425. Kruskal implementation idea.

Sort edges, DSU union if endpoints differ, sum weights until V-1 edges.

Q426. Accounts Merge.

Union-Find over emails (or account indices), group emails by root, sort.

Q427. Redundant Connection.

Union-Find: the first edge connecting already-connected nodes is redundant.

Q428. Number of provinces.

Connected components on adjacency matrix with DFS or DSU.

Q429. Evaluate Division.

Weighted graph a->b with ratio; DFS/BFS multiply weights along a path; or weighted Union-Find.

Q430. Strongly connected components (Kosaraju / Tarjan).

Kosaraju: DFS order, transpose graph, DFS in reverse finishing order. Tarjan: single DFS with low-link values. O(V + E).

Q431. Bridges and articulation points.

Tarjan's DFS with discovery time and low values: edge (u,v) is a bridge if `low[v] > disc[u]`.

Q432. Graph coloring and scheduling exam slots.

Greedy colouring by degree order; exact solution is NP-hard.

Q433. Detect cycle in a directed graph using Kahn's algorithm.

If topological order includes fewer than V nodes, there is a cycle.

Q434. Clone graph code.

```
Node? CloneGraph(Node? node) { if (node == null) return null; var map = new Dictionary<Node, Node>();
    Node Dfs(Node n) { if (map.TryGetValue(n, out var c)) return c; c = new Node(n.val); map[n] = c;
        foreach (var nb in n.neighbors) c.neighbors.Add(Dfs(nb)); return c; }
    return Dfs(node); }
```

Q435. Real-world: how would you detect circular fund transfers (money laundering loops)?

Model accounts as nodes and transfers as directed edges in a time window; find cycles with DFS colouring or SCCs (Tarjan); flag SCCs with more than one account.

30. Dynamic Programming (More)

Q436. Fibonacci: recursion vs memo vs tabulation vs O(1) space.

Recursion O(2^n); memo/tab O(n); two variables O(1) space; matrix exponentiation O(log n).

Q437. Frog jump (min energy with 1 or 2 steps).

`dp[i] = min(dp[i-1] + |h[i]-h[i-1]|, dp[i-2] + |h[i]-h[i-2]|)`.

Q438. Min Cost Climbing Stairs.

`dp[i] = cost[i] + min(dp[i-1], dp[i-2])`; answer min of last two.

Q439. Maximum sum of non-adjacent elements.

Same as House Robber.

Q440. Ninja's training / paint house (no same choice on consecutive days).

`dp[day][colour] = cost + min(dp[day-1][other colours])`.

Q441. Unique Paths with obstacles code.

```
int UniquePathsWithObstacles(int[][] g) { int n = g[0].Length; var dp = new int[n]; dp[0] = 1;
    foreach (var row in g) for (int j = 0; j < n; j++) { if (row[j] == 1) dp[j] = 0; else if (j > 0) dp[
    j] += dp[j - 1]; }
    return dp[n - 1]; }
```

Q442. Minimum Path Sum in a grid.

`dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])`.

Q443. Triangle minimum path sum.

Bottom-up: `dp[j] = tri[i][j] + min(dp[j], dp[j+1])`.

Q444. Minimum/maximum falling path sum.

`dp[r][c] = m[r][c] + min(dp[r-1][c-1], dp[r-1][c], dp[r-1][c+1])`.

Q445. Cherry Pickup II (two robots).

3D DP over (row, col1, col2) with 9 transitions.

Q446. Subset sum equals target.

Boolean DP over sums, iterate target downward per item. O(n * target).

Q447. Count subsets with sum K.

`dp[s] += dp[s - x]` iterating s downward. Careful with zeros.

Q448. Partition into two subsets with minimum difference.

Subset-sum reachable table up to total/2; answer `total - 2 * best`.

Q449. Target Sum (+/- signs).

Convert to count subsets with sum `(total + target) / 2`.

Q450. Unbounded knapsack / rod cutting.

Iterate capacity upward so items can repeat: `dp[w] = max(dp[w], dp[w - wt] + val)`.

Q451. Coin Change II (number of combinations).

```
int Change(int amount, int[] coins) { var dp = new int[amount + 1]; dp[0] = 1;
    foreach (var c in coins) for (int a = c; a <= amount; a++) dp[a] += dp[a - c]; return dp[amount]; }
```

Coins outer loop counts combinations; amount outer loop would count permutations.

Q452. Longest Palindromic Subsequence.

LCS of s and reverse(s), or interval DP `dp[i][j]`.

Q453. Minimum insertions to make a string palindrome.

`n - LPS(s)`.

Q454. Minimum deletions/insertions to convert a to b.

Deletions = `len(a) - LCS`, insertions = `len(b) - LCS`.

Q455. Shortest Common Supersequence.

Length `m + n - LCS`; reconstruct by walking the LCS table.

Q456. Distinct Subsequences (count of t in s).

`dp[i][j] = dp[i-1][j] + (s[i-1]==t[j-1] ? dp[i-1][j-1] : 0)`.

Q457. Wildcard Matching (? and *).

`dp[i][j]`: if p[j-1]=='*' then `dp[i-1][j] || dp[i][j-1]`; if '?' or equal then `dp[i-1][j-1]`.

Q458. Regular Expression Matching (. and *).

* case: zero occurrences `dp[i][j-2]` or one more if preceding char matches `dp[i-1][j]`.

Q459. Interleaving String.

`dp[i][j]` true if s3 prefix of length i+j formed by s1[0..i) and s2[0..j).

Q460. Longest Increasing Subsequence - print the sequence.

O(n^2) DP with a parent array to reconstruct; or patience sorting with predecessor indices.

Q461. Number of LIS.

Track length and count arrays; combine counts when lengths tie.

Q462. Largest Divisible Subset.

Sort; LIS-style DP with divisibility, parent pointers.

Q463. Longest String Chain.

Sort by length; for each word try removing each char and look up dp of predecessor.

Q464. Russian Doll Envelopes.

Sort by width asc and height desc; LIS on heights with binary search. O(n log n).

Q465. Matrix Chain Multiplication.

Interval DP: `dp[i][j] = min over k (dp[i][k] + dp[k+1][j] + p[i-1]*p[k]*p[j])`. O(n^3).

Q466. Burst Balloons.

Interval DP choosing the last balloon to burst in (i, j): `dp[i][j] = max(nums[i]*nums[k]*nums[j] + dp[i][k] + dp[k][j])`.

Q467. Minimum cost to cut a stick.

Sort cuts plus ends; interval DP over cut positions.

Q468. Palindrome Partitioning II (minimum cuts).

Precompute palindrome table; `cuts[i] = min(cuts[j-1] + 1)` for palindromic s[j..i].

Q469. Boolean parenthesization (count ways to evaluate True).

Interval DP storing true/false counts for each substring.

Q470. Egg dropping problem.

`dp[e][f]` minimum trials; optimised version: `dp[m][e] = dp[m-1][e-1] + dp[m-1][e] + 1` floors covered with m moves.

Q471. Maximal Square in a binary matrix.

`dp[i][j] = 1 + min(top, left, top-left)` if cell is 1; answer max^2.

Q472. Count square submatrices with all ones.

Same DP, sum all dp values.

Q473. Best time to buy and sell stock IV (k transactions).

`buy[j] = max(buy[j], sell[j-1] - p)`, `sell[j] = max(sell[j], buy[j] + p)` for j in 1..k.

Q474. Decode Ways code.

```
int NumDecodings(string s) { int prev2 = 1, prev1 = s[0] == '0' ? 0 : 1;
    for (int i = 1; i < s.Length; i++) { int cur = s[i] != '0' ? prev1 : 0;
        int two = (s[i - 1] - '0') * 10 + (s[i] - '0'); if (two >= 10 && two <= 26) cur += prev2;
        prev2 = prev1; prev1 = cur; }
    return prev1; }
```

Q475. Jump Game II (minimum jumps).

Greedy BFS levels: track current end and farthest; increment jumps when reaching current end. O(n).

Q476. Perfect Squares (min squares summing to n).

`dp[i] = min(dp[i - j*j] + 1)`. O(n sqrt n).

Q477. Integer Break.

DP or math: break into as many 3s as possible.

Q478. Paint Fence (no more than two adjacent same).

`same = diff_prev; diff = (same_prev + diff_prev) * (k - 1)`.

Q479. Counting bits / DP on bitmasks idea.

Bitmask DP stores state as subsets (e.g. travelling salesman `dp[mask][i]`), O(2^n * n^2).

Q480. Travelling Salesman Problem (bitmask DP).

`dp[mask][i]` minimum cost visiting set mask ending at i; transition to unvisited j. O(2^n n^2), feasible for n <= 20.

Q481. Digit DP idea.

Count numbers up to N with a property by processing digits with states (position, tight, other state). Used for "count numbers with sum of digits = k".

Q482. How do you identify a DP problem?

Asks for count/min/max/possible over choices, and brute force recursion repeats the same subproblems. Write recursion first, add memo, then convert to tabulation and optimise space.

Q483. Memoisation in C#: dictionary vs array?

Arrays are much faster when the state space is small integers; dictionaries for sparse or composite states (use value tuples as keys).

31. Greedy (More)

Q484. Activity selection / maximum meetings in one room.

Sort by end time; pick each meeting whose start >= last end.

Q485. Fractional knapsack.

Sort by value/weight ratio; take items fully then a fraction of the next. O(n log n).

Q486. Minimum number of coins (canonical system like Indian currency).

Greedy picking the largest denomination works for canonical systems (1, 2, 5, 10, ...).

Q487. Job sequencing with deadlines.

Sort by profit desc; place each job in the latest free slot before its deadline (DSU for speed).

Q488. Assign cookies.

Sort both; two pointers giving the smallest sufficient cookie to each child.

Q489. Lemonade change.

Track $5 and $10 notes; give change preferring $10+$5 for $20.

Q490. Minimum arrows to burst balloons.

Sort by end; shoot at the end of the first; skip balloons that start before the arrow.

Q491. Valid Parenthesis String with '*'.

Track range [low, high] of open counts; * can be (,), or empty; ensure high never negative and low reaches 0.

Q492. Queue reconstruction by height.

Sort by height desc, k asc; insert each person at index k.

Q493. Minimum number of taps to water a garden / video stitching.

Convert to intervals, then greedy jump game style covering.

Q494. Huffman coding.

Min-heap of frequencies; merge two smallest repeatedly to build the tree; optimal prefix codes.

Q495. Largest number from array of numbers.

Sort strings with comparator `(b + a).CompareTo(a + b)`; handle all zeros.

Q496. Boats, platforms and meeting rooms - common greedy theme?

Sort, then make the locally optimal choice that keeps the most room for future choices (earliest end, pair extremes).

Q497. Split array into consecutive subsequences.

Count map and "need" map: extend existing subsequence ending at x-1 if possible, else start new with x, x+1, x+2.

Q498. Gas Station code.

```
int CanCompleteCircuit(int[] gas, int[] cost) { int total = 0, tank = 0, start = 0;
    for (int i = 0; i < gas.Length; i++) { int d = gas[i] - cost[i]; total += d; tank += d; if (tank < 0)
    { start = i + 1; tank = 0; } }
    return total < 0 ? -1 : start; }
```

32. Math and Bit Manipulation (More)

Q499. GCD and LCM.

`Gcd(a, b) = b == 0 ? a : Gcd(b, a % b)`; `Lcm = a / Gcd(a, b) * b`. O(log min).

Q500. Sieve of Eratosthenes.

```
bool[] Sieve(int n) { var comp = new bool[n + 1];
    for (int i = 2; (long)i * i <= n; i++) if (!comp[i]) for (int j = i * i; j <= n; j += i) comp[j] =
    true;
    return comp; }   // primes are i >= 2 with !comp[i]
```

O(n log log n).

Q501. Count primes less than n.

Sieve and count.

Q502. Prime factorisation.

Divide by i from 2 while i*i <= n; leftover > 1 is prime. O(sqrt n).

Q503. Check Armstrong number.

Sum of digits each raised to number of digits equals the number.

Q504. Reverse an integer with overflow check.

Build result with long or check before multiplying: if result > int.MaxValue / 10 return 0.

Q505. Palindrome number without converting to string.

Reverse half of the number and compare.

Q506. Count digits / sum of digits.

Loop `n /= 10`; or `(int)Math.Log10(n) + 1` for positive n.

Q507. Trailing zeros in n factorial.

Sum `n/5 + n/25 + n/125 + ...`.

Q508. Modular exponentiation and why mod 1e9+7?

Fast power with `% mod` at each step to avoid overflow. 1e9+7 is a large prime keeping results in int range with safe multiplication in long.

Q509. nCr mod p.

Precompute factorials and inverse factorials using Fermat's little theorem (`inv(x) = x^(p-2) mod p`).

Q510. Excel column title to number and back.

Number: `r = r * 26 + (c - 'A' + 1)`. Title: `n--, char('A' + n % 26), n /= 26` repeated.

Q511. Count set bits (Brian Kernighan).

```
int CountBits(int n) { int c = 0; while (n != 0) { n &= n - 1; c++; } return c; }
```

Q512. Check if the i-th bit is set; toggle; clear the lowest set bit.

`(n >> i) & 1`; `n ^ (1 << i)`; `n & (n - 1)`.

Q513. Isolate the rightmost set bit.

`n & -n`.

Q514. Power of four.

Power of two and the set bit at an even position: `n > 0 && (n & (n - 1)) == 0 && (n & 0x55555555) != 0`.

Q515. Divide two integers without * / %.

Subtract shifted divisor (doubling) from dividend; handle sign and int.MinValue overflow.

Q516. Sum of two integers without + operator.

```
int GetSum(int a, int b) { while (b != 0) { int carry = (a & b) << 1; a ^= b; b = carry; } return a; }
```

Q517. Gray code.

`i ^ (i >> 1)` for i in 0..2^n-1.

Q518. Subsets using bitmask.

For mask from 0 to 2^n - 1, include element i if bit i is set.

Q519. Minimum bit flips to convert a to b.

PopCount(a ^ b).

Q520. XOR of numbers from 1 to n in O(1).

Based on n % 4: n, 1, n+1, 0.

Q521. Maximum XOR of two numbers in an array.

Binary trie of numbers; for each number greedily choose the opposite bit. O(32n).

Q522. Check if a number is even/odd without modulo.

`(n & 1) == 0`.

Q523. How is the ISO 8583 bitmap parsed?

Primary bitmap of 64 bits (16 hex chars); bit 1 indicates a secondary bitmap. For field i (1-based), check `(bitmap >> (64 - i)) & 1`.

```
bool HasField(ulong bitmap, int field) => ((bitmap >> (64 - field)) & 1UL) == 1UL;
var bmp = Convert.ToUInt64("7230001108C18000", 16); HasField(bmp, 2); // true (PAN present)
```

33. Design Data Structures

Q524. Design a HashMap from scratch.

Array of buckets with linked lists (or arrays) of key-value pairs; hash function `key % size`; resize when load factor > 0.75 by rehashing.

Q525. Design a HashSet.

Same as HashMap without values, or a bit array for small integer ranges.

Q526. Design an LFU Cache.

Maps: key -> (value, freq), freq -> ordered list of keys (LinkedHashSet-like), track minFreq. All operations O(1).

Q527. Design a Twitter feed (getNewsFeed top 10).

Per-user tweet lists with timestamps, follow sets; merge recent tweets from followees with a heap (k-way merge).

Q528. Design a hit counter (hits in last 5 minutes).

Circular arrays of size 300 storing timestamp and count per second; sum valid buckets.

Q529. Design an in-memory file system.

Trie of path segments; nodes are directories (children map) or files (content).

Q530. Design a snapshot array.

Per index list of (snapId, value); get via binary search on snapId.

Q531. Design an autocomplete system.

Trie with sentence frequencies; for each prefix, collect top 3 by frequency (cache at nodes or DFS with heap).

Q532. Design a leaderboard.

Dictionary of player scores plus a SortedSet/SortedDictionary by score for top K queries in O(log n).

Q533. Design a stack with increment operation.

Array stack plus a lazy increment array: increment `inc[k-1]`; on pop propagate to the element below.

Q534. Design a max stack.

Two stacks (values and max so far), or a doubly linked list + TreeMap for popMax in O(log n).

Q535. Design an underground system (average travel time).

Map check-ins by id; map (start, end) to (total, count).

Q536. Design a rate limiter with sliding window log.

Per user queue of timestamps; drop older than window; allow if count < limit.

Q537. Design a consistent hashing ring.

Sorted ring of hashed node positions (with virtual nodes) in a SortedDictionary; key maps to the first node clockwise. Used for distributed caches/sharding.

Q538. Design a bloom filter.

Bit array + k hash functions; adds set k bits; queries return "possibly present" if all set, "definitely absent" otherwise. False positives possible, no false negatives. Used to skip lookups for non-existent keys.